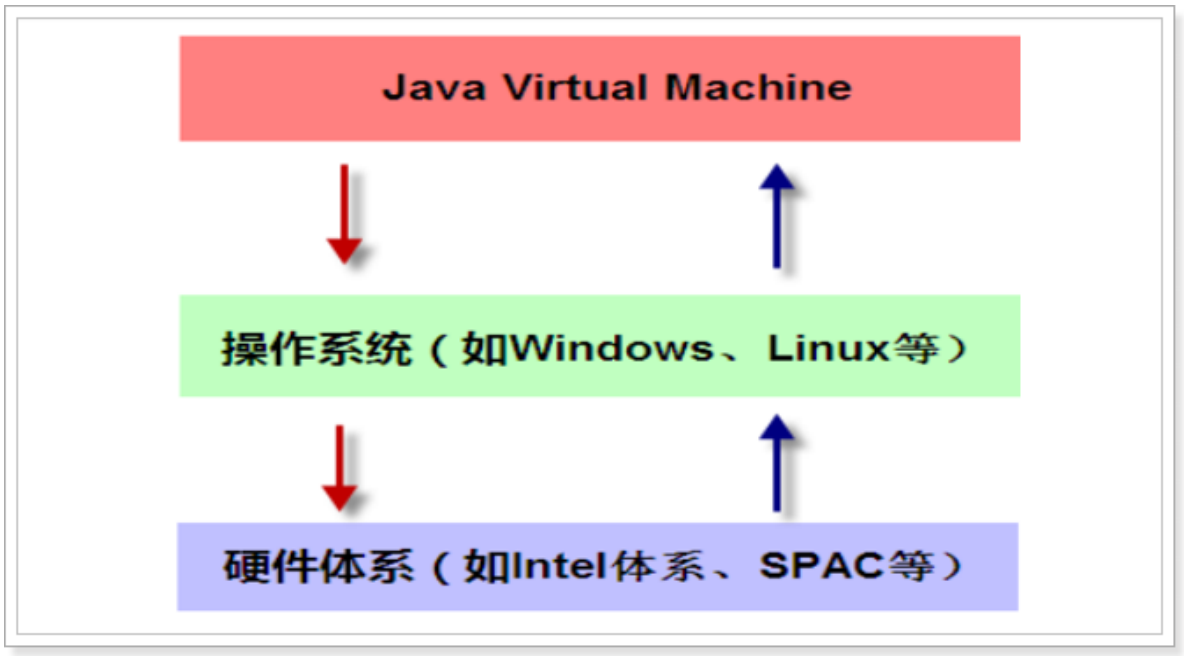
1. JVM快速入门

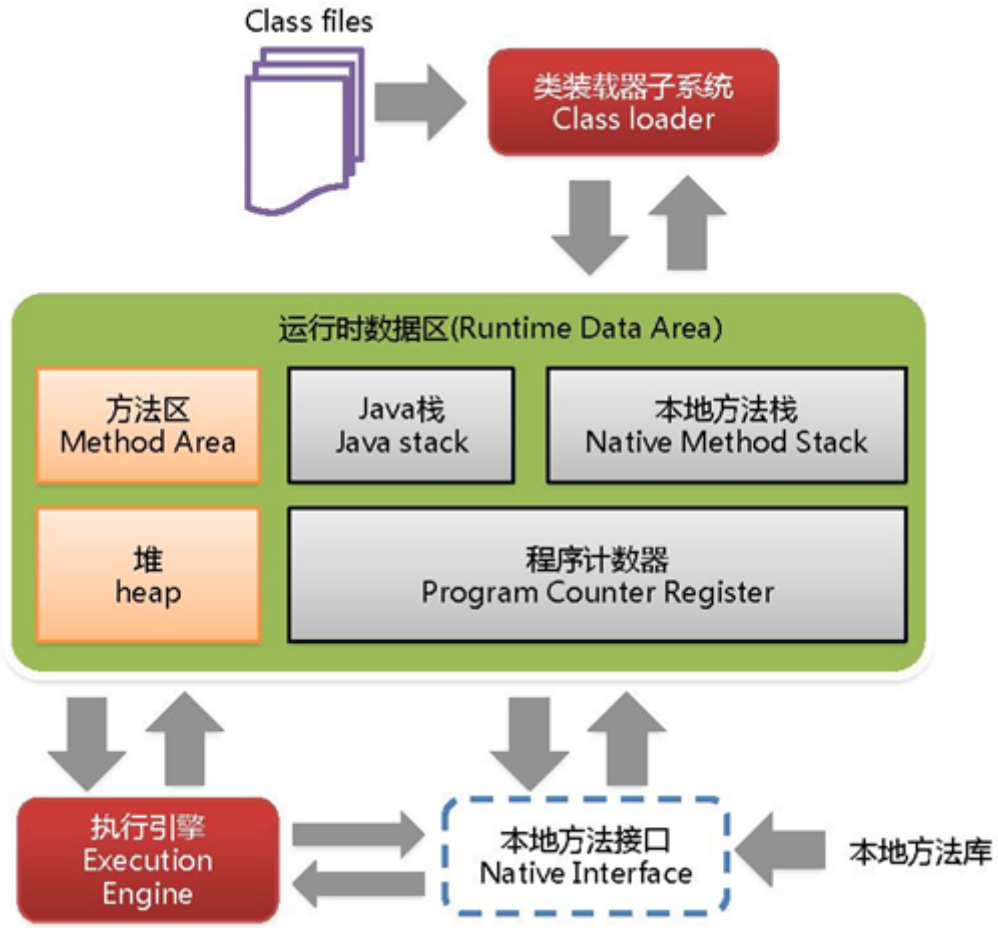
从面试开始：

1. 请谈谈你对JVM 的理解？java8 的虚拟机有什么更新？
2. 什么是OOM ？什么是StackOverflowError？有哪些方法分析？
3. JVM 的常用参数调优你知道哪些？
4. 内存快照抓取和MAT分析DUMP文件知道吗？
5. 谈谈JVM中，对类加载器你的认识？

位置：**JVM**是运行在操作系统之上的，它与硬件没有直接的交互



1.1. 结构图



方法区：存储已**被虚拟机加载的类元数据信息**(元空间)

堆：**存放对象实例**，几乎所有的对象实例都在这里分配内存

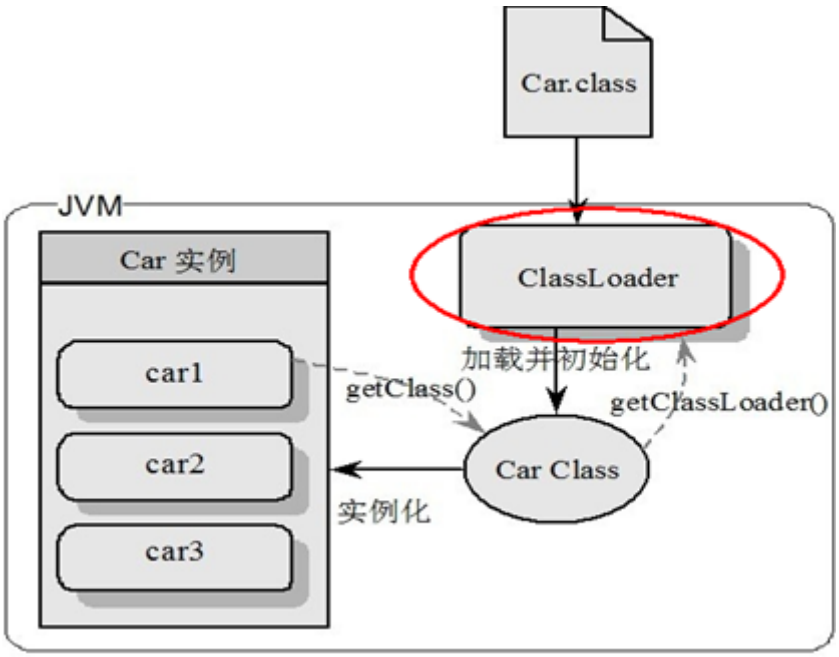
虚拟机栈：虚拟机栈描述的是**Java方法执行的内存模型**，每个方法被执行的时候都会同时创建一个**栈帧**（Stack Frame）用于存储局部变量表、操作栈、动态链接、方法出口等信息

程序计数器：当前线程所执行的字节码的**行号指示器**

本地方法栈：本地方法栈则是为虚拟机使用到的**Native方法服务**。

1.2. 类加载器ClassLoader

负责加载class文件，class文件在文件开头有特定的文件标示，并且ClassLoader只负责class文件的加载，至于它是否可以运行，则由Execution Engine决定。



类加载器分为四种：前三种为虚拟机自带的加载器。

- 启动类加载器（Bootstrap）C++

  负责加载$JAVA_HOME中jre/lib/**rt.jar**里所有的class，由C++实现，不是ClassLoader子类

- 扩展类加载器（Extension）Java

  负责加载java平台中**扩展功能**的一些jar包，包括$JAVA_HOME中jre/lib/*.jar或-Djava.ext.dirs指定目录下的jar包

  简单记忆就是：负责加载jre中除了rt.jar以外的jar包

- 应用程序类加载器（AppClassLoader）Java

  也叫系统类加载器，负责加载**classpath**中指定的jar包及目录中class

- 用户自定义加载器 Java.lang.ClassLoader的子类(基类)，用户可以定制类的加载方式

工作过程：

- 1、当AppClassLoader加载一个class时，它首先不会自己去尝试加载这个类，而是把类加载请求委派给父类加载器ExtClassLoader去完成。
- 2当ExtClassLoader加载一个class时，它首先也不会自己去尝试加载这个类，而是把类加载请求委派给BootStrapClassLoader去完成。
- 3、如果BootStrapClassLoader加载失败（例如在$JAVA_HOME/jre/lib里未查找到该class），会使用ExtClassLoader来尝试加载；
- 4若ExtClassLoader也加载失败，则会使用AppClassLoader来加载

- 5、如果AppClassLoader也加载失败，则会报出异常ClassNotFoundException

其实这就是所谓的**双亲委派模型**。简单来说：如果一个类加载器收到了类加载的请求，它首先不会自己去尝试加载这个类，而是把**请求委托给父加载器去完成，依次向上**。

双亲委托模式：保证jdk的稳定型 安全性

好处：**防止内存中出现多份同样的字节码**(安全性角度)

写段儿代码演示类加载器：

```
public class Demo {

    public Demo() {
        super();
    }

    public static void main(String[] args) {
        Object obj = new Object();
        String s = new String();
        Demo demo = new Demo();
        System.out.println(obj.getClass().getClassLoader());//获取哪个类加载器加载的，为null就是BootStrpt加载的
        System.out.println(s.getClass().getClassLoader());

 System.out.println(demo.getClass().getClassLoader().getParent().getParent());

 System.out.println(demo.getClass().getClassLoader().getParent());
        System.out.println(demo.getClass().getClassLoader());
    }
}
```

打印控制台中的sun.misc.Launcher，是一个java虚拟机的入口应用

1.3. 执行引擎Execution Engine

Execution Engine执行引擎负责解释命令，提交操作系统执行。

1.4. 本地接口Native Interface

本地接口的作用是融合不同的编程语言为 Java 所用，它的初衷是融合 C/C++程序，Java 诞生的时候是 C/C++横行的时候，要想立足，必须有调用 C/C++程序，于是就在内存中专门开辟了一块区域处理标记为native的代码，它的具体做法是 Native Method Stack中登记 native方法，在Execution Engine 执行时加载native libraies。

目前该方法使用的越来越少了，除非是与硬件有关的应用，比如通过Java程序驱动打印机或者Java系统管理生产设备，在企业级应用中已经比较少见。因为现在的异构领域间的通信很发达，比如可以使用 Socket通信，也可以使用Web Service等等，不多做介绍。

1.5. Native Method Stack

它的具体做法是Native Method Stack中登记native方法，在Execution Engine 执行时加载本地方法库。

1.6. PC寄存器

每个线程都有一个程序计数器，是**线程私有的**，就是一个指针，指向方法区中的方法字节码（**用来存储指向下一条指令的地址**，即 将要执行的指令代码），由执行引擎读取下一条指令，是一个非常小的内存空间，几乎可以忽略不记。

1.7. Method Area方法区

方法区是被所有线程共享，所有字段和方法字节码，以及一些特殊方法如构造函数，接口代码也在此定义。简单说，所有定义的方法的信息都保存在该区域，**此区属于共享区间。**

静态变量+常量+类信息(构造方法/接口定义)+运行时常量池存在方法区中

But

实例变量存在堆内存中,和方法区无关

2. stack栈

Stack 栈是什么?

栈也叫栈内存，**主管Java程序的运行，是在线程创建时创建**，它的生命期是跟随线程的生命期，线程结束栈内存也就释放，**对于栈来说不存在垃圾回收问题**，只要线程一结束该栈就Over，生命周期和线程一致，是线程私有的。**8种基本类型的变量+对象的引用变量+实例方法**都是在函数的栈内存中分配。

栈存储什么?

栈中的数据都是以**栈帧**（Stack Frame）的格式存在，栈帧是一个内存区块，是一个数据集，是一个有关方法(Method)和运行期数据的数据集。

栈帧中主要保存3 类数据：

- 本地变量（Local Variables）：输入参数和输出参数以及方法内的变量。
- 栈操作（Operand Stack）：记录出栈、入栈的操作。
- 栈帧数据（Frame Data）：包括类文件、方法等等。

栈运行原理：

当一个方法A被调用时就产生了一个栈帧 F1，并被压入到栈中

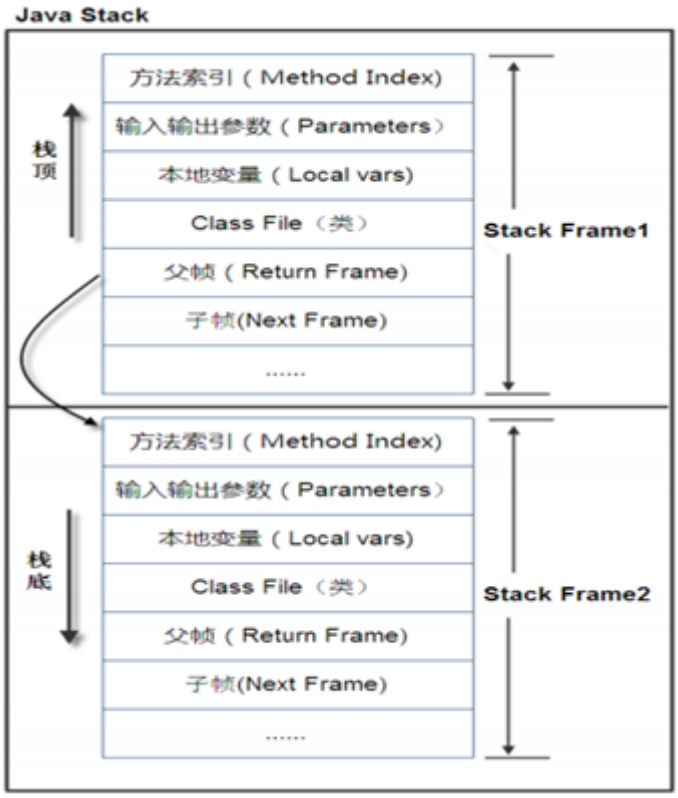
A方法又调用了 B方法，于是产生栈帧 F2 也被压入栈，

B方法又调用了 C方法，于是产生栈帧 F3 也被压入栈，

……

执行完毕后，先弹出F3栈帧，再弹出F2栈帧，再弹出F1栈帧……

遵循“先进后出”或者“后进先出”原则。



图示在一个栈中有两个栈帧：

栈帧 2是最先被调用的方法，先入栈，

然后方法 2 又调用了方法1，栈帧 1处于栈顶的位置，

栈帧 2 处于栈底，执行完毕后，依次弹出栈帧 1和栈帧 2，

线程结束，栈释放。

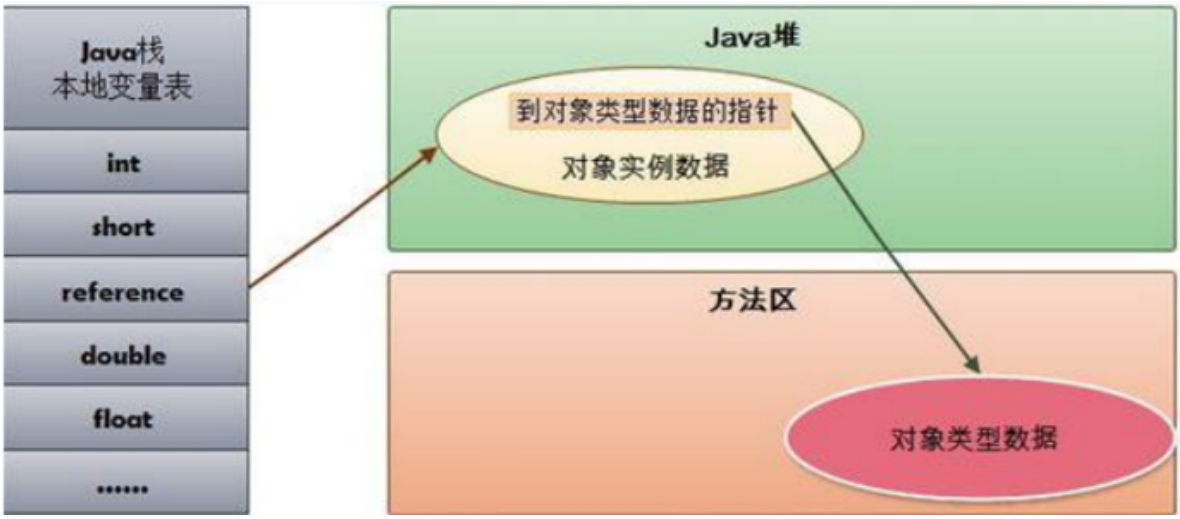
每执行一个方法都会产生一个栈帧，保存到栈(后进先出)的**顶部，顶部栈就是当前的方法，该方法执行完毕 后会自动将此栈帧出栈。**

常见问题栈溢出：Exception in thread "main" java.lang.StackOverflowError

通常出现在递归调用时。

3. 堆

堆栈方法区的关系：



HotSpot是使用指针的方式来访问对象：

- Java堆中会存放访问**类元数据**的地址
- reference存储的就是对象的地址

三种JVM：

•Sun公司的**HotSpot**

•BEA公司的**JRockit**

•IBM公司的**J9 VM**

3.1. 堆体系概述

Java7之前

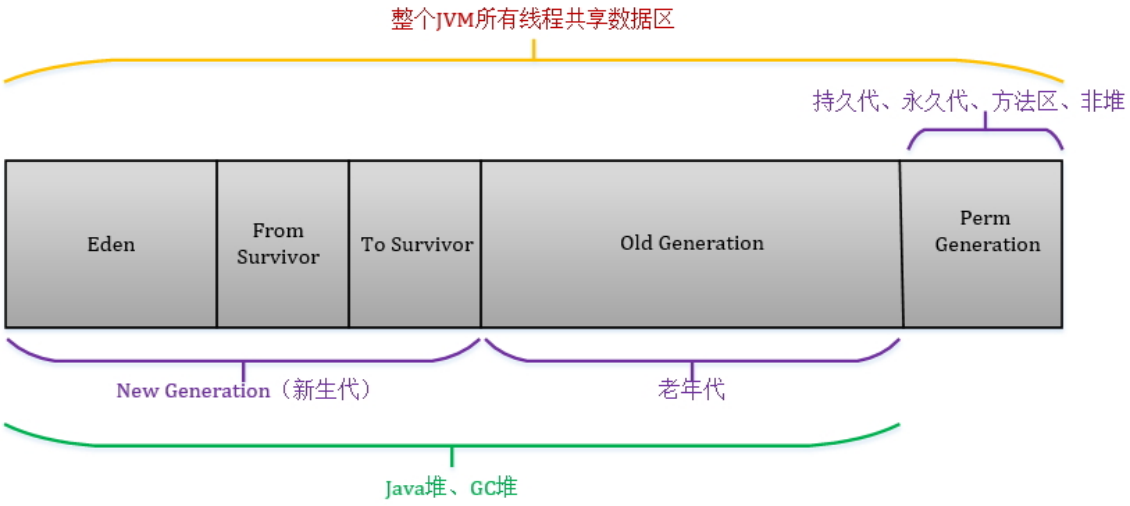
Heap 堆：一个JVM实例只存在一个堆内存，堆内存的大小是可以调节的。类加载器读取了类文件后，需要把类、方法、常变量放到堆内存中，保存所有引用类型的真实信息，以方便执行器执行，堆内存逻辑上分为三部分：

- Young Generation Space 新生区 Young/New
- Tenure generation space 养老区 Old/Tenure

- Permanent Space 永久区 Perm

也称为：新生代（年轻代）、老年代、永久代（持久代）。

其中JVM堆分为新生代和老年代



3.1.1. 新生区

新生区是对象的诞生、成长、消亡的区域，一个对象在这里产生，应用，最后被垃圾回收器收集，结束生命。新生区又分为两部分： 伊甸区（Eden space）和幸存者区（Survivor pace），所有的对象都是在伊甸区被new出来的。幸存区有两个： 0区（Survivor 0 space）和1区（Survivor 1 space）。当伊甸园的空间用完时，程序又需要创建对象，JVM的垃圾回收器将对伊甸园区进行垃圾回收(**Minor GC**)，将伊甸园区中的不再被其他对象所引用的对象进行销毁。然后将伊甸园中的剩余对象移动到幸存 0区。若幸存 0区也满了，再对该区进行垃圾回收，然后移动到 1 区。那如果1区也满了呢？再次垃圾回收，满足条件后再移动到养老区。若养老区也满了，那么这个时候将产生**MajorGC（FullGC）**，进行养老区的内存清理。若养老区执行了Full GC之后发现依然无法进行对象的保存，就会产生OOM异常“OutOfMemoryError”。

如果出现java.lang.OutOfMemoryError: Java heap space异常，说明Java虚拟机的堆内存不够。原因有二：

（1）Java虚拟机的堆内存设置不够，可以通过参数-Xms、-Xmx来调整。

（2）代码中创建了大量大对象，并且长时间不能被垃圾收集器收集（存在被引用）。

3.1.2. 老年代

经历多次GC仍然存在的对象（默认是15次），老年代的对象比较稳定，不会频繁的GC

3.1.3. 永久代

永久存储区是一个常驻内存区域，用于存放JDK自身所携带的 Class,Interface 的元数据，也就是说它存储的是运行环境必须的类信息，被装载进此区域的数据是不会被垃圾回收器回收掉的，关闭 JVM 才会释放此区域所占用的内存。

如果出现java.lang.OutOfMemoryError: PermGen space，**说明是Java虚拟机对永久代Perm内存设置不够。一般出现这种情况，都是程序启动需要加载大量的第三方jar包。例如：在一个Tomcat下部署了太多的应用。或者大量动态反射生成的类不断被加载，最终导致Perm区被占满。**

Jdk1.6及之前： 有永久代，常量池1.6在方法区

Jdk1.7： 有永久代，但已经逐步“去永久代”，常量池1.7在堆

Jdk1.8及之后： 无永久代，常量池1.8在元空间（Metaspace）

实际而言，方法区（Method Area）和堆一样，是各个线程共享的内存区域，它用于存储虚拟机加载的：类信息+普通常量+静态常量+编译器编译后的代码等等，**虽然JVM规范将方法区描述为堆的一个逻辑部分，但它却还有一个别名叫做Non-Heap(非堆)，目的就是要和堆分开。**

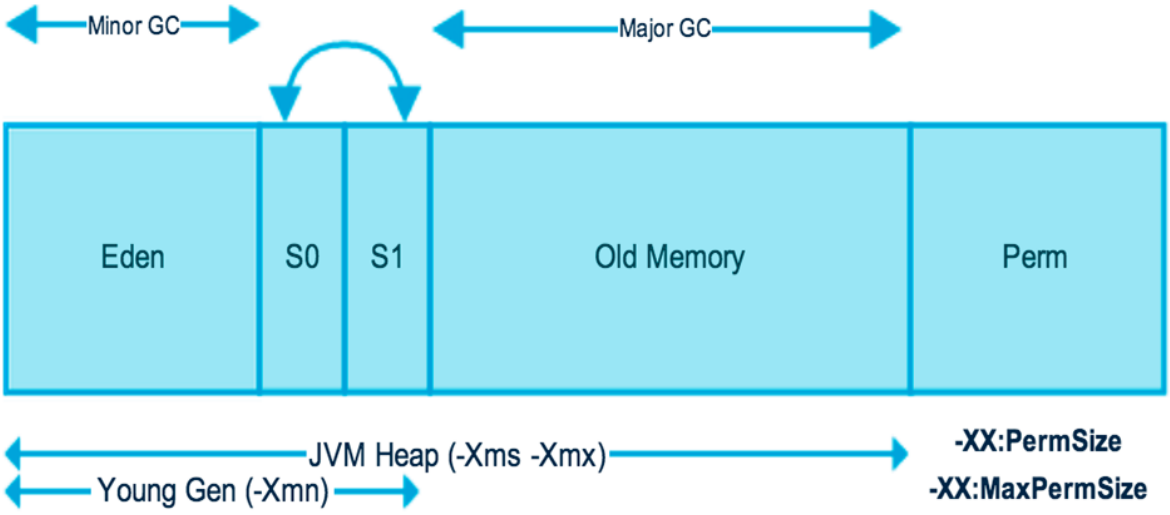
对于HotSpot虚拟机，很多开发者习惯将方法区称之为“永久代(Parmanent Gen)”，但严格本质上说两者不同，或者说使用永久代来实现方法区而已，永久代是方法区(相当于是一个接口interface)的一个实现，**jdk1.7的版本中，已经将原本放在永久代的字符串常量池移走。**

常量池（Constant Pool）是方法区的一部分，Class文件除了有类的版本、字段、方法、接口等描述信息外，还有一项信息就是常量池，这部分内容将在类加载后进入方法区的运行时常量池中存放。

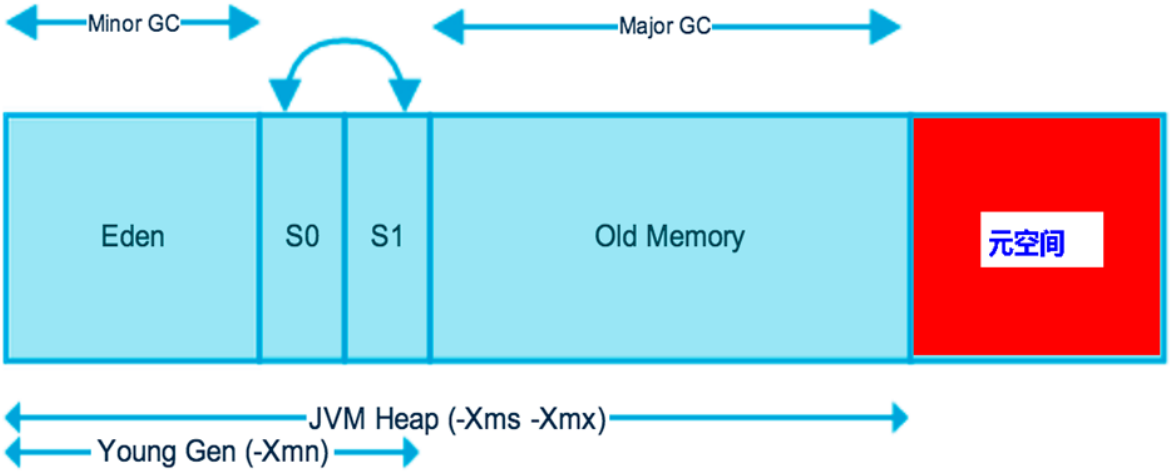
3.2. 堆参数调优入门

均以JDK1.8+HotSpot为例

jdk1.7：



jdk1.8：



3.2.1. 常用JVM参数

怎么对jvm进行调优？通过**参数配置**

参数	备注
-Xms	初始堆大小。只要启动，就占用的堆大小，默认是内存的1/64
-Xmx	最大堆大小。默认是内存的1/4

参数	备注
-Xmn	新生区堆大小
-XX:+PrintGCDetails	输出详细的GC处理日志

java代码查看jvm堆的默认值大小：

```
Runtime.getRuntime().maxMemory()    // 堆的最大值，默认是内存的1/4
Runtime.getRuntime().totalMemory()  // 堆的当前总大小，默认是内存的1/64
```

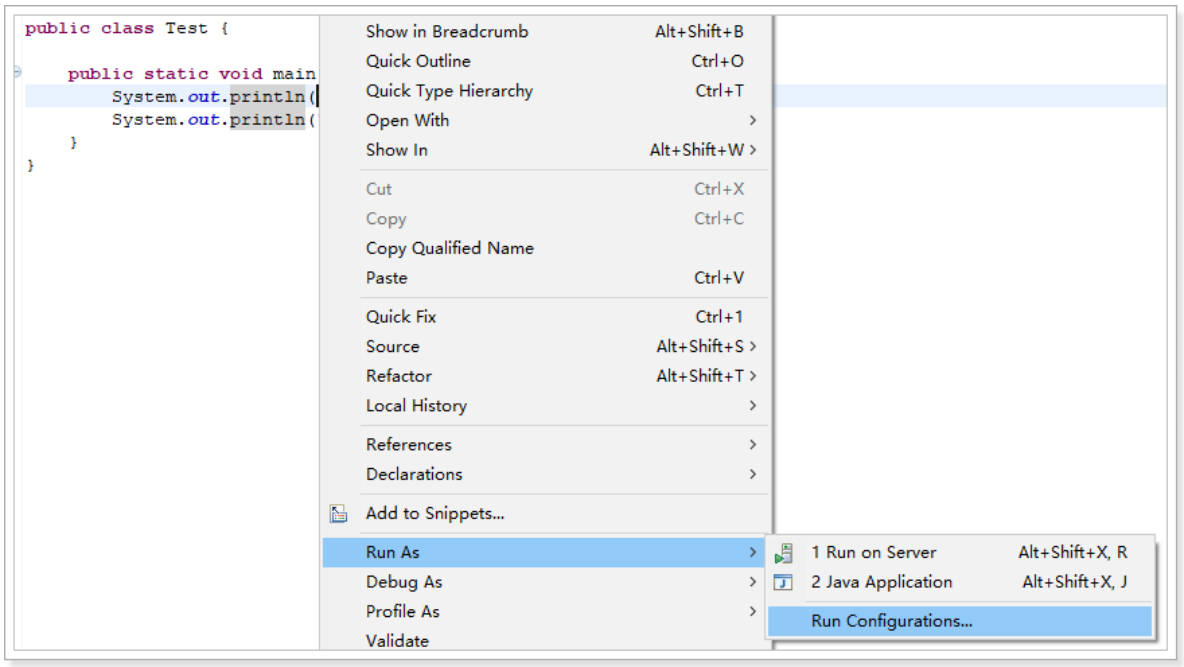
3.2.2. 怎么设置JVM参数

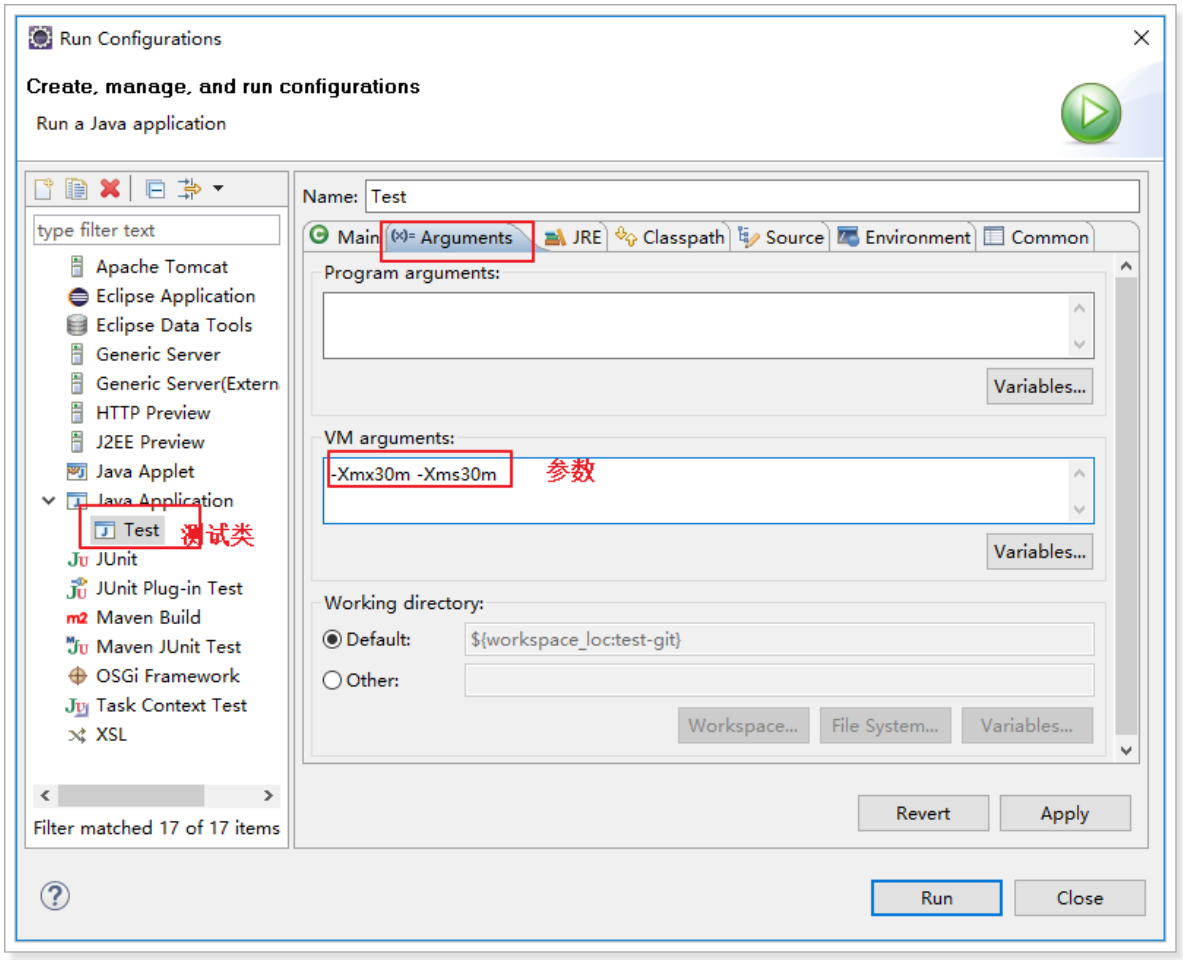
程序运行时，可以给该程序设置jvm参数，不同的工具设置方式不同。

如果是命令行运行：

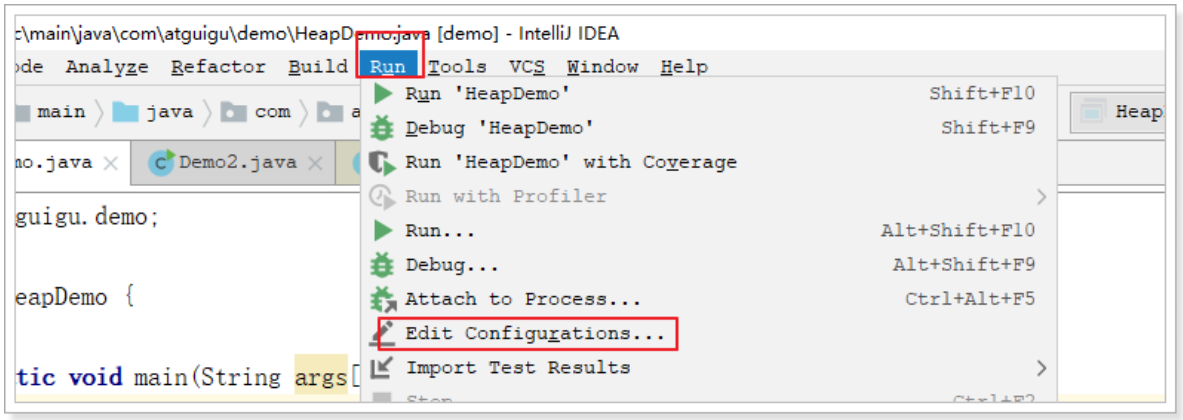
```
java -Xmx50m -Xms10m HeapDemo
```

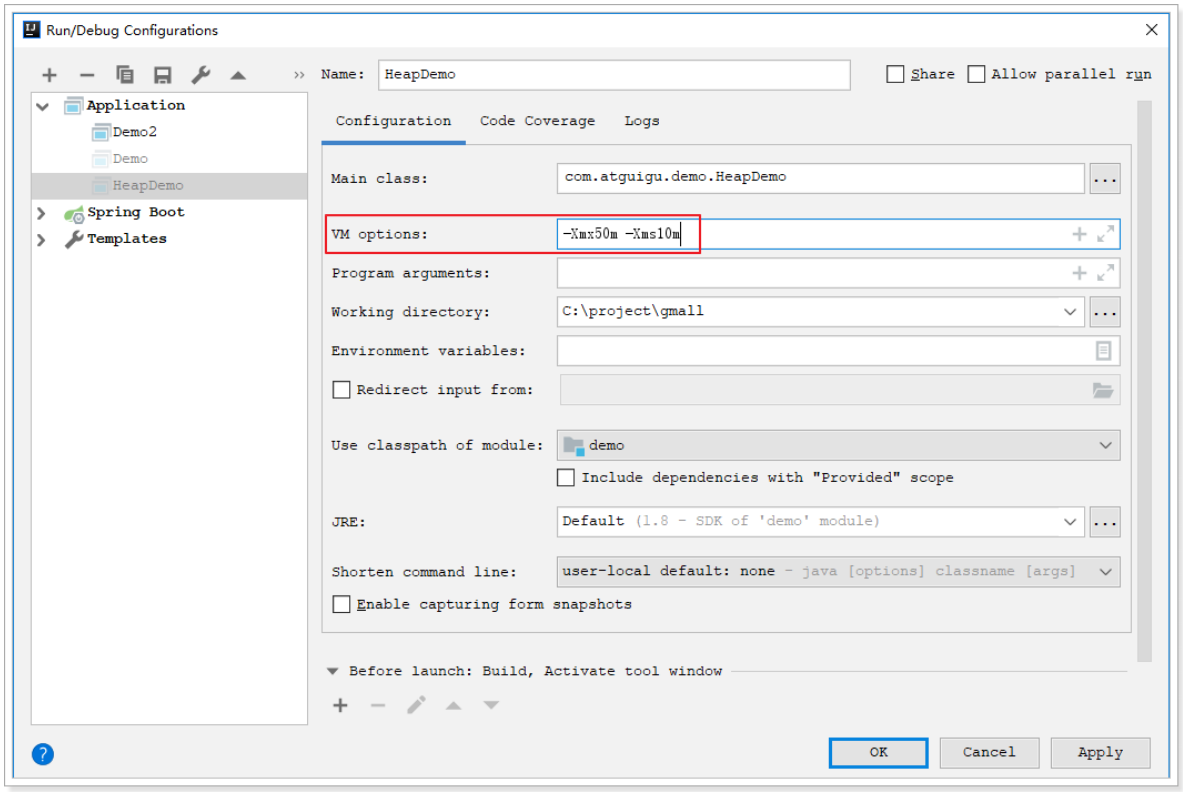
eclipse运行的设置方式如下：





idea运行时设置方式如下：





3.2.3. 查看堆内存详情

```
public class Demo2 {
    public static void main(String[] args) {

        System.out.print("最大堆大小：");
        System.out.println(Runtime.getRuntime().maxMemory() / 1024.0
/ 1024 + "M");
        System.out.print("当前堆大小：");
        System.out.println(Runtime.getRuntime().totalMemory() /
1024.0 / 1024 + "M");

 System.out.println("================================================
==");

        byte[] b = null;
        for (int i = 0; i < 10; i++) {
            b = new byte[1 * 1024 * 1024];
        }
    }
}
```

执行前配置参数：-Xmx50m -Xms30m -XX:+PrintGCDetails

执行：看到如下信息

```
最大堆大小：43.5M
当前堆大小：27.5M
==================================================
Heap
 PSYoungGen      total 17920K, used 13340K [0x00000000fec00000, 0x0000000100000000, 0x0000000100000000)
  eden space 15360K, 86% used [0x00000000fec00000,0x00000000ff9071f0,0x00000000ffb00000)
  from space 2560K, 0% used [0x00000000ffd80000,0x00000000ffd80000,0x0000000100000000)
  to   space 2560K, 0% used [0x00000000ffb00000,0x00000000ffb00000,0x00000000ffd80000)
 ParOldGen       total 10240K, used 0K [0x00000000fce00000, 0x00000000fd800000, 0x00000000fec00000)
  object space 10240K, 0% used [0x00000000fce00000,0x00000000fce00000,0x00000000fd800000)
 Metaspace       used 3486K, capacity 4496K, committed 4864K, reserved 1056768K
  class space    used 377K, capacity 388K, committed 512K, reserved 1048576K
```

新生代和老年代的堆大小之和是Runtime.getRuntime().totalMemory()

3.2.4. GC演示

```java
public class HeapDemo {

    public static void main(String args[]) {


 System.out.println("=====================Begin=====================
===");
        System.out.print("最大堆大小：Xmx=");
        System.out.println(Runtime.getRuntime().maxMemory() / 1024.0
/ 1024 + "M");

        System.out.print("剩余堆大小：free mem=");
        System.out.println(Runtime.getRuntime().freeMemory() / 1024.0
/ 1024 + "M");

        System.out.print("当前堆大小：total mem=");
        System.out.println(Runtime.getRuntime().totalMemory() /
1024.0 / 1024 + "M");

        System.out.println("==================First
Allocated==================");
        byte[] b1 = new byte[5 * 1024 * 1024];
        System.out.println("5MB array allocated");

        System.out.print("剩余堆大小：free mem=");
```

```
        System.out.println(Runtime.getRuntime().freeMemory() / 1024.0
/ 1024 + "M");

        System.out.print("当前堆大小: total mem=");
        System.out.println(Runtime.getRuntime().totalMemory() /
1024.0 / 1024 + "M");

        System.out.println("=================Second
Allocated===================");
        byte[] b2 = new byte[10 * 1024 * 1024];
        System.out.println("10MB array allocated");

        System.out.print("剩余堆大小: free mem=");
        System.out.println(Runtime.getRuntime().freeMemory() / 1024.0
/ 1024 + "M");

        System.out.print("当前堆大小: total mem=");
        System.out.println(Runtime.getRuntime().totalMemory() /
1024.0 / 1024 + "M");


 System.out.println("=====================OOM=======================
=");
        System.out.println("OOM!!!");
        System.gc();
        byte[] b3 = new byte[40 * 1024 * 1024];
    }
}
```

jvm参数设置成最大堆内存100M，当前堆内存10M： -Xmx100m -Xms10m -XX:+PrintGCDetails

再次运行，可以看到minor GC和full GC日志：

```
[GC (System.gc()) [PSYoungGen: 948K->488K(2560K)] 16610K->16253K(20480K), 0.0018261 secs] [Times: user=0.00 s
[Full GC (System.gc()) [PSYoungGen: 488K->0K(2560K)] [ParOldGen: 15765K->16148K(17920K)] 16253K->16148K(20480
Heap
 PSYoungGen      total 2560K, used 61K [0x00000000fdf00000, 0x00000000fe200000, 0x0000000100000000)
  eden space 2048K, 3% used [0x00000000fdf00000,0x00000000fdf0f748,0x00000000fe100000)
  from space 512K, 0% used [0x00000000fe180000,0x00000000fe180000,0x00000000fe200000)
  to   space 512K, 0% used [0x00000000fe100000,0x00000000fe100000,0x00000000fe180000)
 ParOldGen       total 59392K, used 57108K [0x00000000f9c00000, 0x00000000fd600000, 0x00000000fdf00000)
```

3.2.5. OOM演示

把上面案例中的jvm参数改成最大堆内存设置成50M，当前堆内存设置成10M，执行测试： -Xmx50m -Xms10m

```
=====================Begin=========================
最大堆大小：Xmx=44.5M
剩余堆大小：free mem=8.186859130859375M
当前堆大小：total mem=9.5M
=================First Allocated=====================
5MB array allocated
剩余堆大小：free mem=3.1868438720703125M
当前堆大小：total mem=9.5M
================Second Allocated====================
10MB array allocated
剩余堆大小：free mem=3.68682861328125M
当前堆大小：total mem=20.0M
=====================OOM=========================
OOM!!!
Exception in thread "main" java.lang.OutOfMemoryError: Java heap
space
    at com.atguigu.demo.HeapDemo.main(HeapDemo.java:40)
```

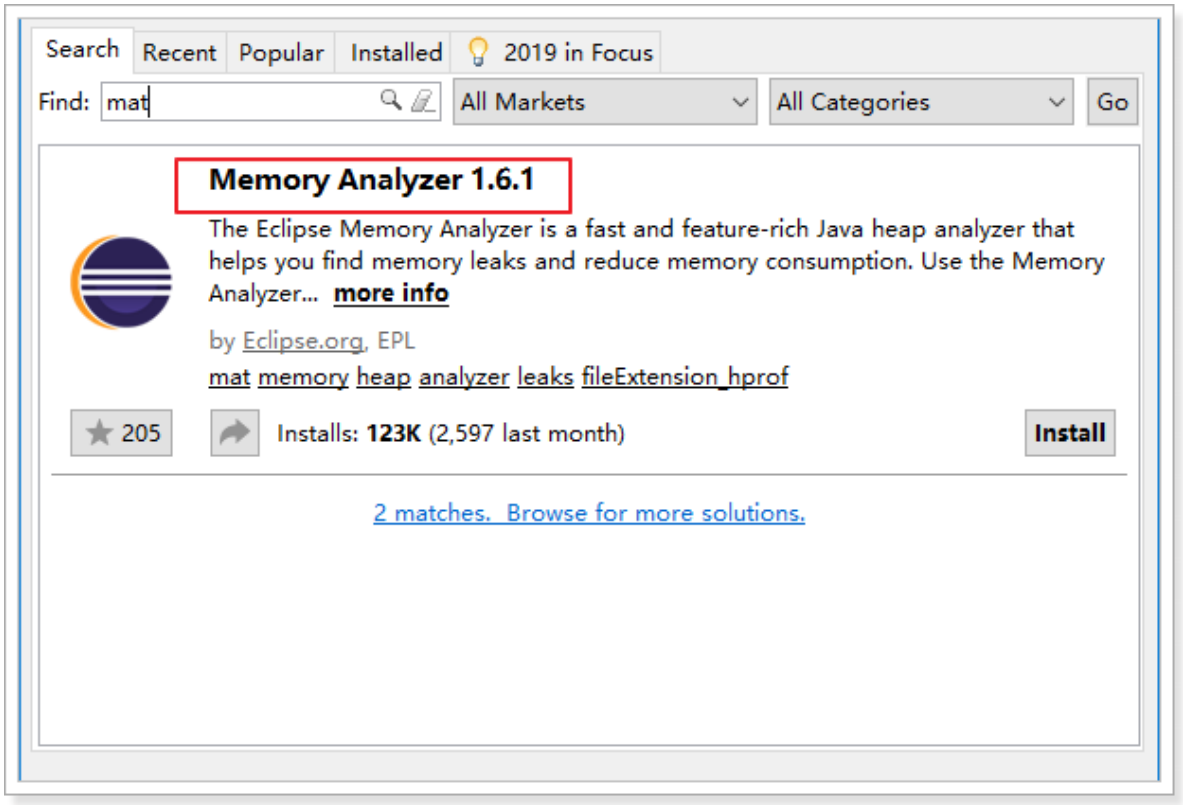
实际开发中怎么定位这种错误信息？MAT工具

3.3. MAT工具

MAT(Eclipse Memory Analyzer)

- ✓ 分析dump文件，快速定位内存泄露；
- ✓ 获得堆中对象的统计数据
- ✓ 获得对象相互引用的关系
- ✓ 采用树形展现对象间相互引用的情况
- ✓ 支持使用OQL语言来查询对象信息

安装方式：eclipse插件市场下载

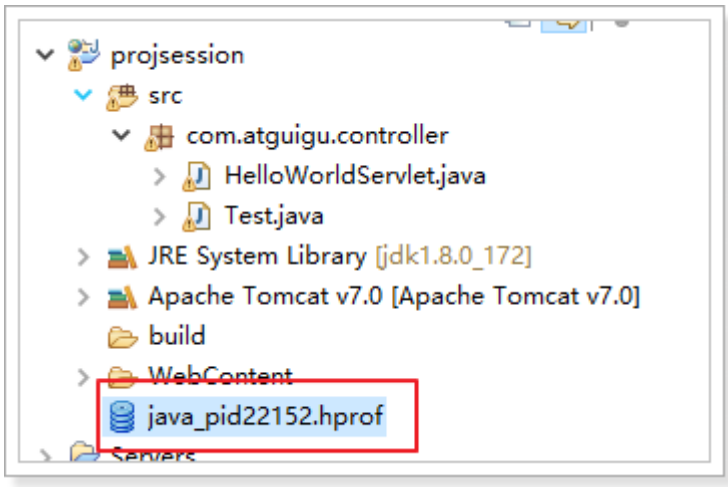


3.3.1. MAT工具的使用

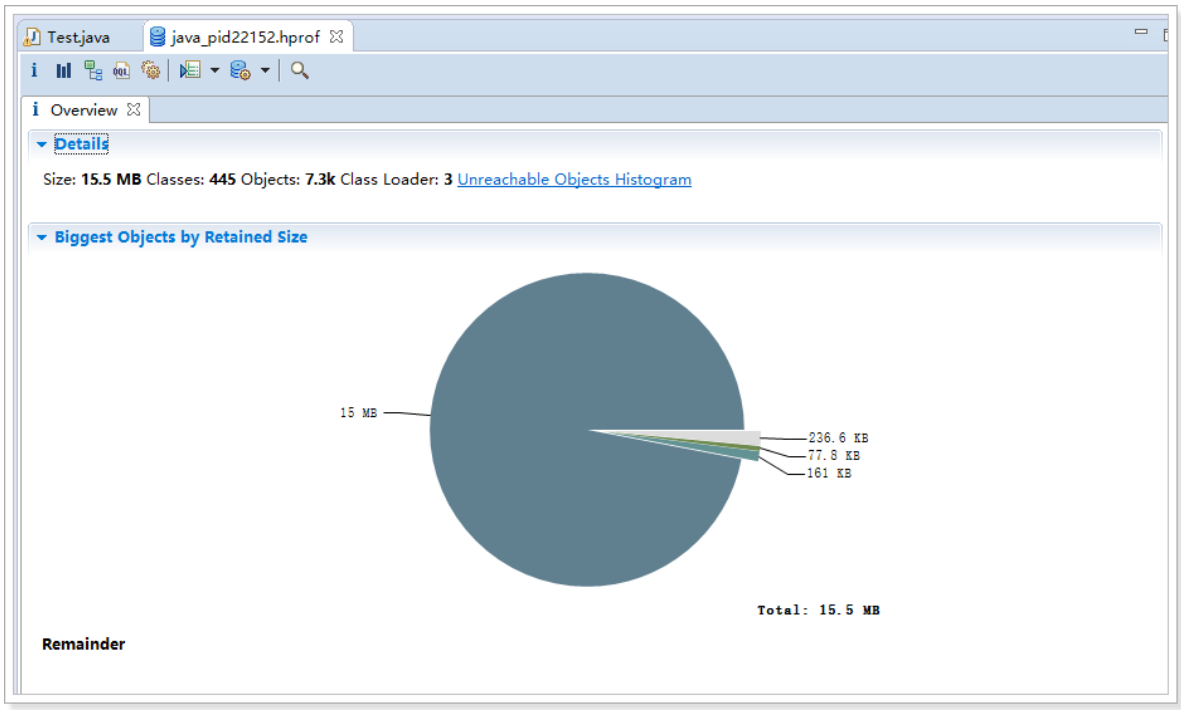
运行参数：-Xmx30m -Xms10m -XX:+HeapDumpOnOutOfMemoryError

```
当前堆大小：total mem=20.0M
=====================OOM=========================
OOM!!!
java.lang.OutOfMemoryError: Java heap space
Dumping heap to java_pid22152.hprof ...
Heap dump file created [17018721 bytes in 0.028 secs]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at com.atguigu.controller.Test.main(Test.java:40)
```

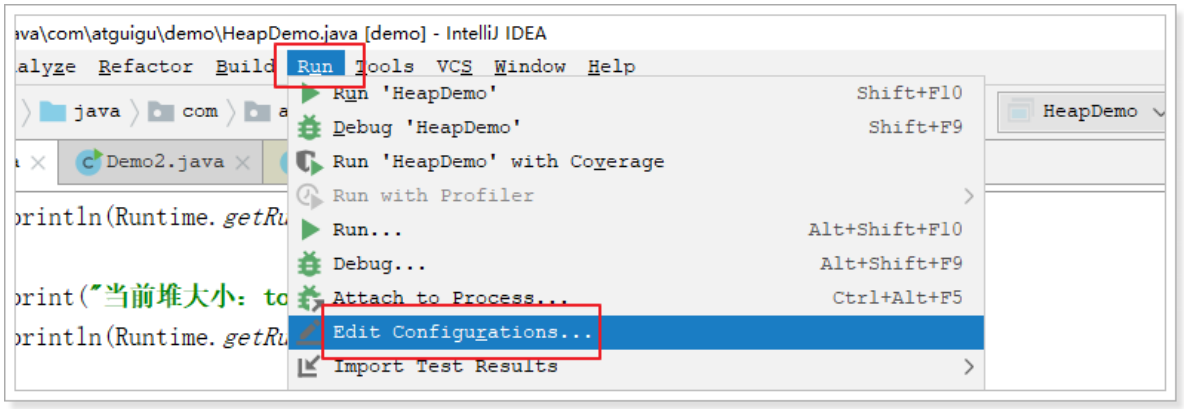
重新刷新项目：看到dump文件



打开：



3.3.2. idea分析dump文件

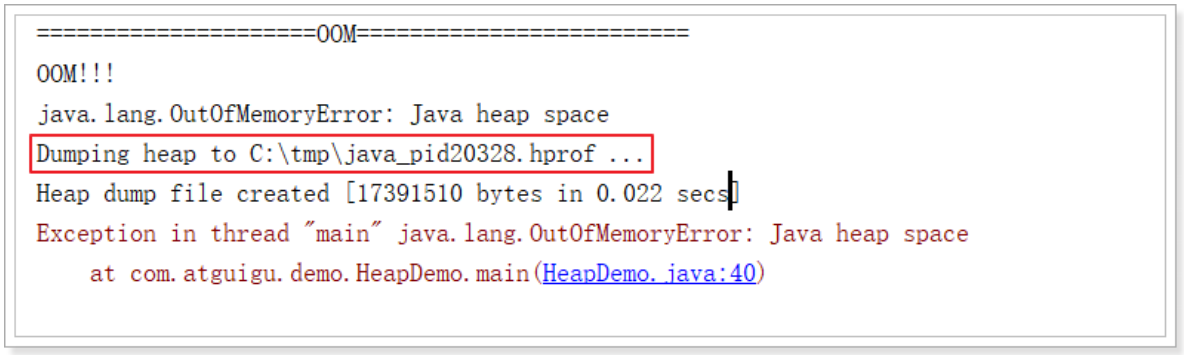


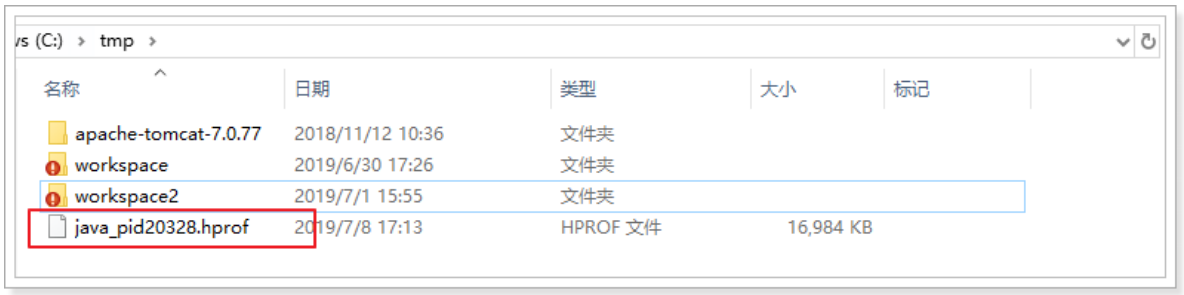
把上例中运行参数改成：

```
-Xmx50m -Xms10m -XX:+HeapDumpOnOutOfMemoryError -
XX:HeapDumpPath=D:\tmp
```

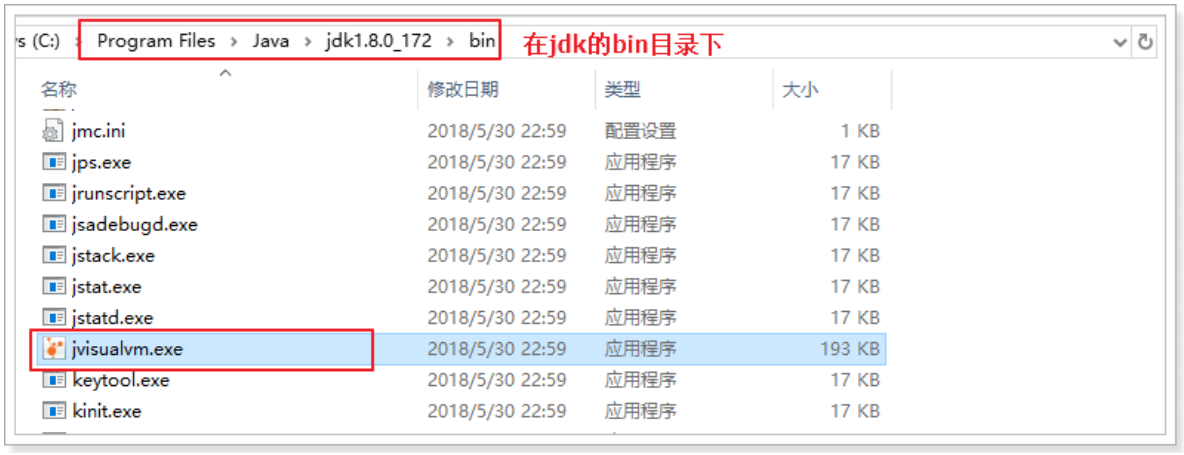
-XX:HeapDumpPath：生成dump文件路径。

再次执行：生成C:\tmp\java_pid20328.hprof文件

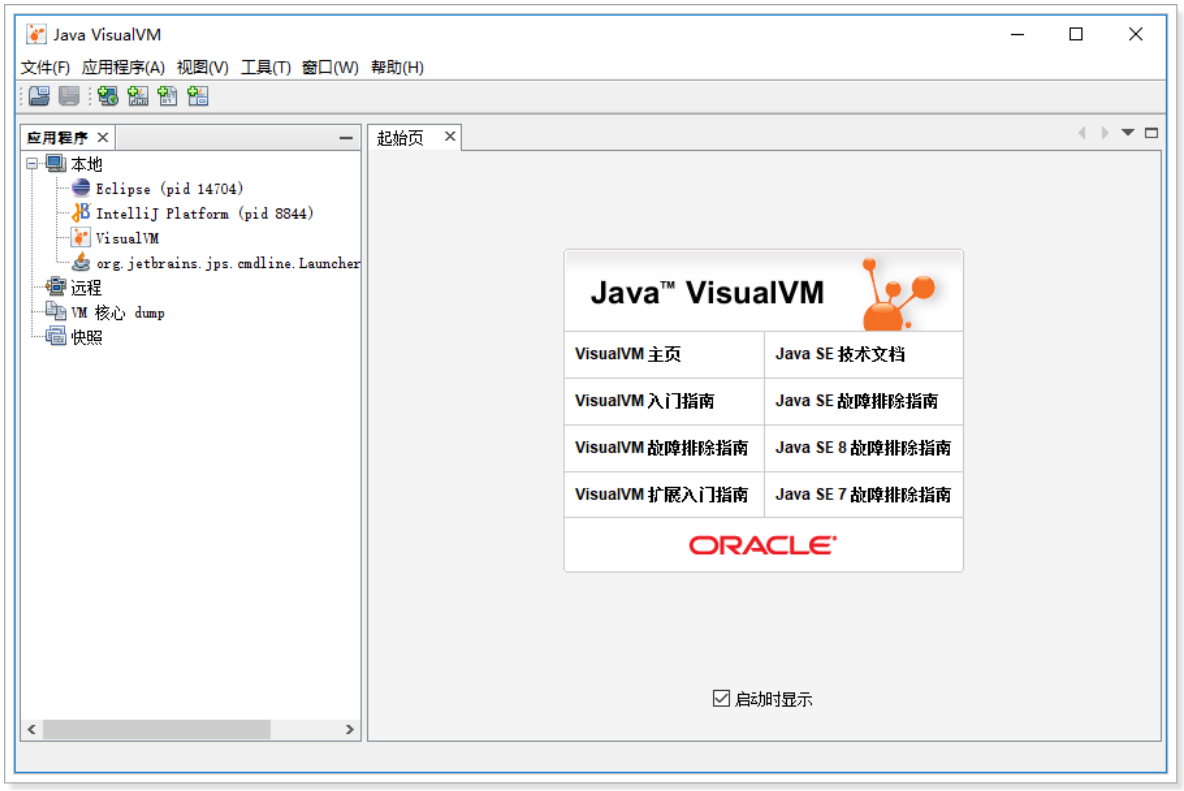




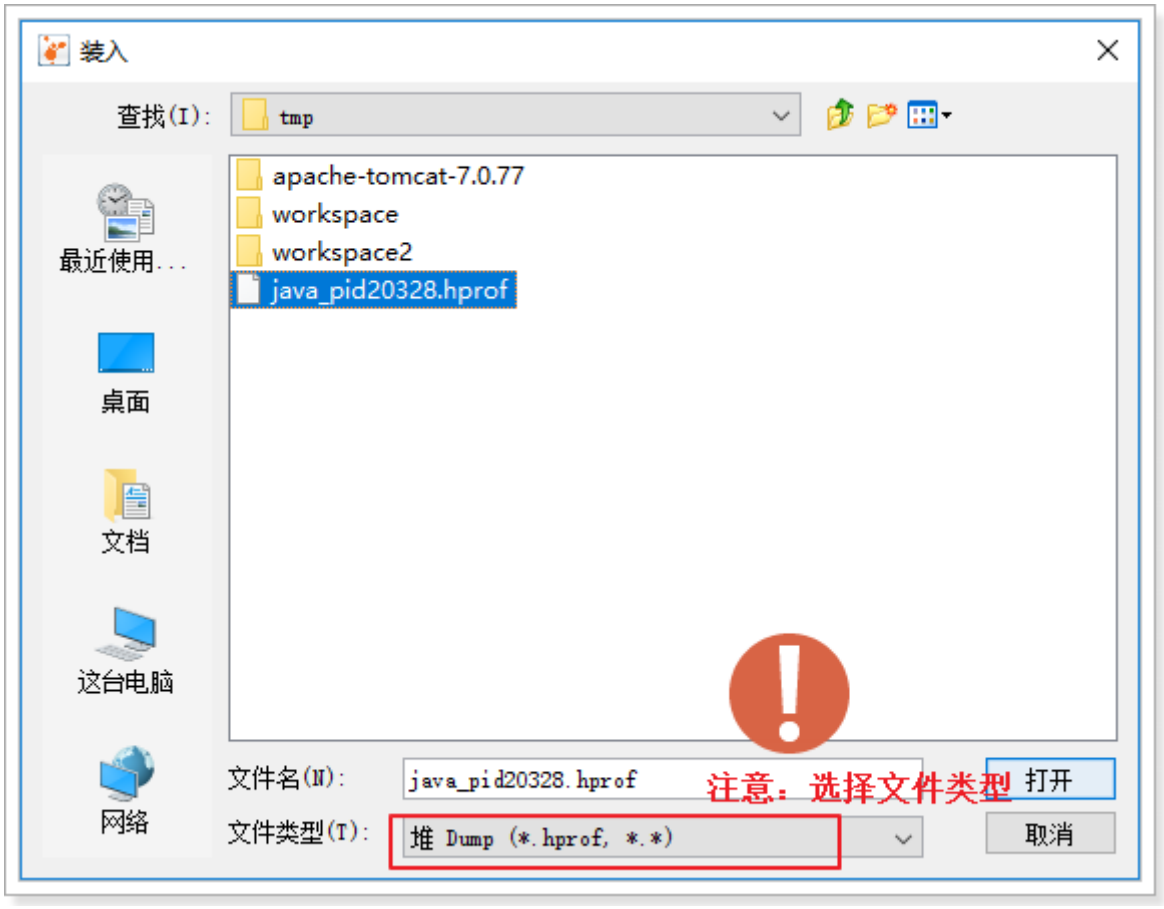
生成的这个文件怎么打开？jdk自带了该类型文件的解读工具：**jvisualvm.exe**



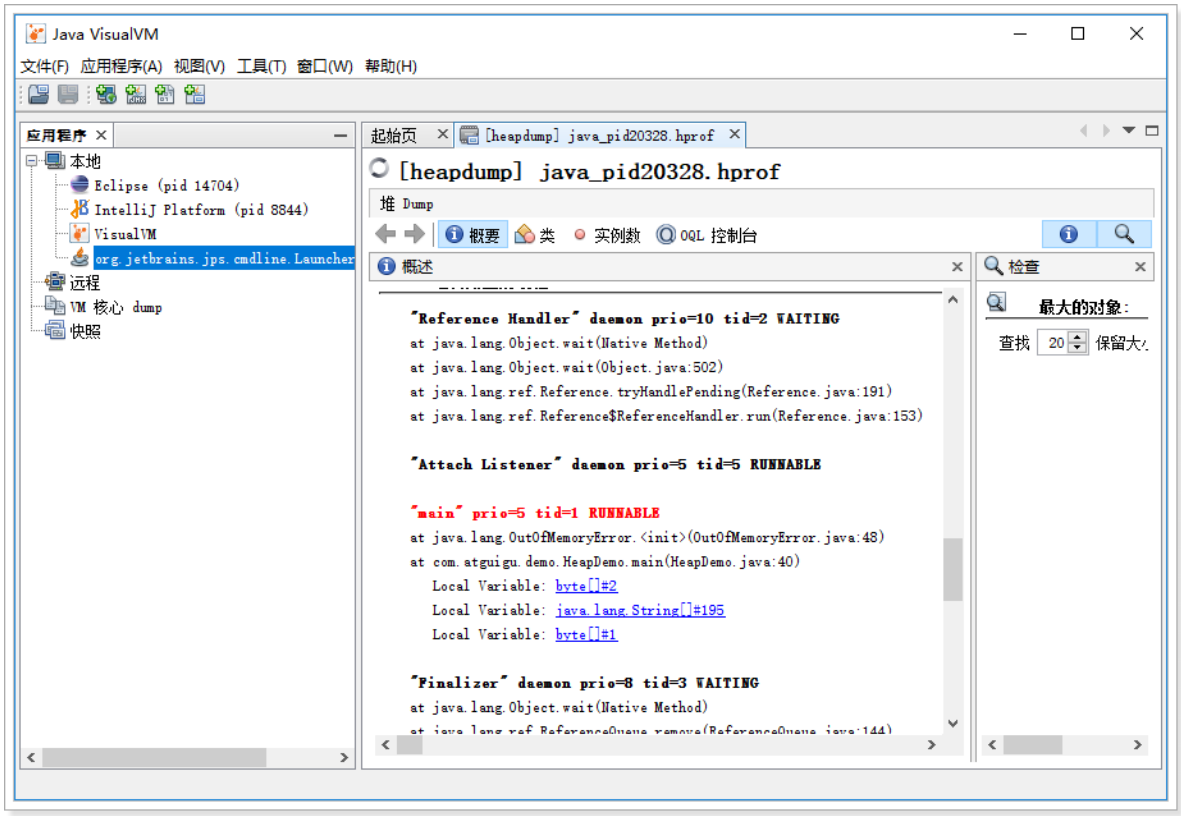
双击打开：



文件-->装入-->选择要打开的文件即可


装入
查找(I):
tmp
apache-tomcat-7.0.77
workspace
workspace2
java_pid20328.hprof
最近使用...
桌面
文档
这台电脑
网络
文件名(N):
java_pid20328.hprof
注意：选择文件类型
打开
文件类型(T):
堆 Dump (*.hprof, *.*)
取消

装入后:


Java VisualVM
文件(F) 应用程序(A) 视图(V) 工具(T) 窗口(W) 帮助(H)
应用程序
本地
Eclipse (pid 14704)
IntelliJ Platform (pid 8844)
VisualVM
org.jetbrains.jps.cmdline.Launcher
远程
VM 核心 dump
快照
起始页
[heapdump] java_pid20328.hprof
[heapdump] java_pid20328.hprof
堆 Dump
概要
类
实例数
OQL 控制台
概述
检查
最大的对象:
"Reference Handler" daemon prio=10 tid=2 WAITING
at java.lang.Object.wait(Native Method)
at java.lang.Object.wait(Object.java:502)
at java.lang.ref.Reference.tryHandlePending(Reference.java:191)
at java.lang.ref.Reference$ReferenceHandler.run(Reference.java:153)
"Attach Listener" daemon prio=5 tid=5 RUNNABLE
"main" prio=5 tid=1 RUNNABLE
at java.lang.OutOfMemoryError.<init>(OutOfMemoryError.java:48)
at com.atguigu.demo.HeapDemo.main(HeapDemo.java:40)
Local Variable: byte[]#2
Local Variable: java.lang.String[]#195
Local Variable: byte[]#1
"Finalizer" daemon prio=8 tid=3 WAITING
at java.lang.Object.wait(Native Method)

4. GC垃圾回收

面试题：

- JVM内存模型以及分区，需要详细到每个区放什么
- 堆里面的分区：Eden，survival from to，老年代，各自的特点。
- GC的三种收集方法：标记清除、标记整理、复制算法的原理与特点，分别用在什么地方
- Minor GC与Full GC分别在什么时候发生

JVM垃圾判定算法：（对象已死？）

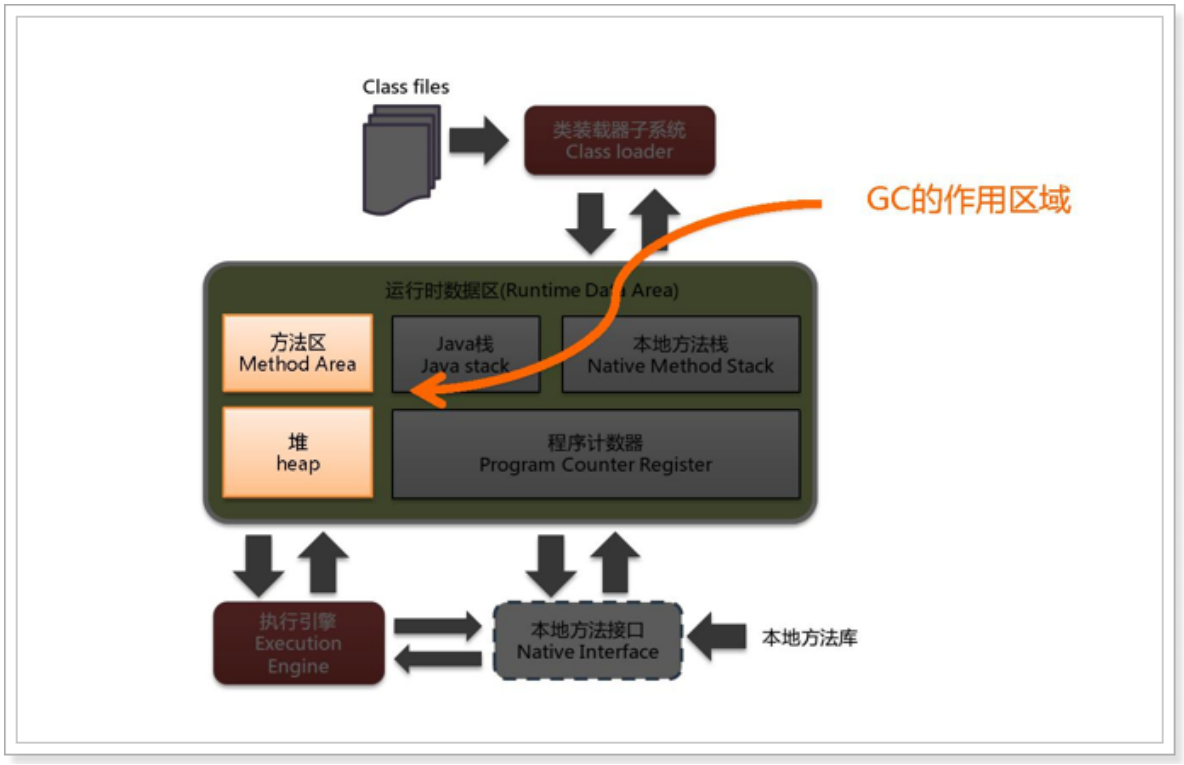
- 引用计数法(Reference-Counting)
- 可达性分析算法（根搜索算法）

GC垃圾回收主要有四大算法：（怎么找到已死对象并清除？）

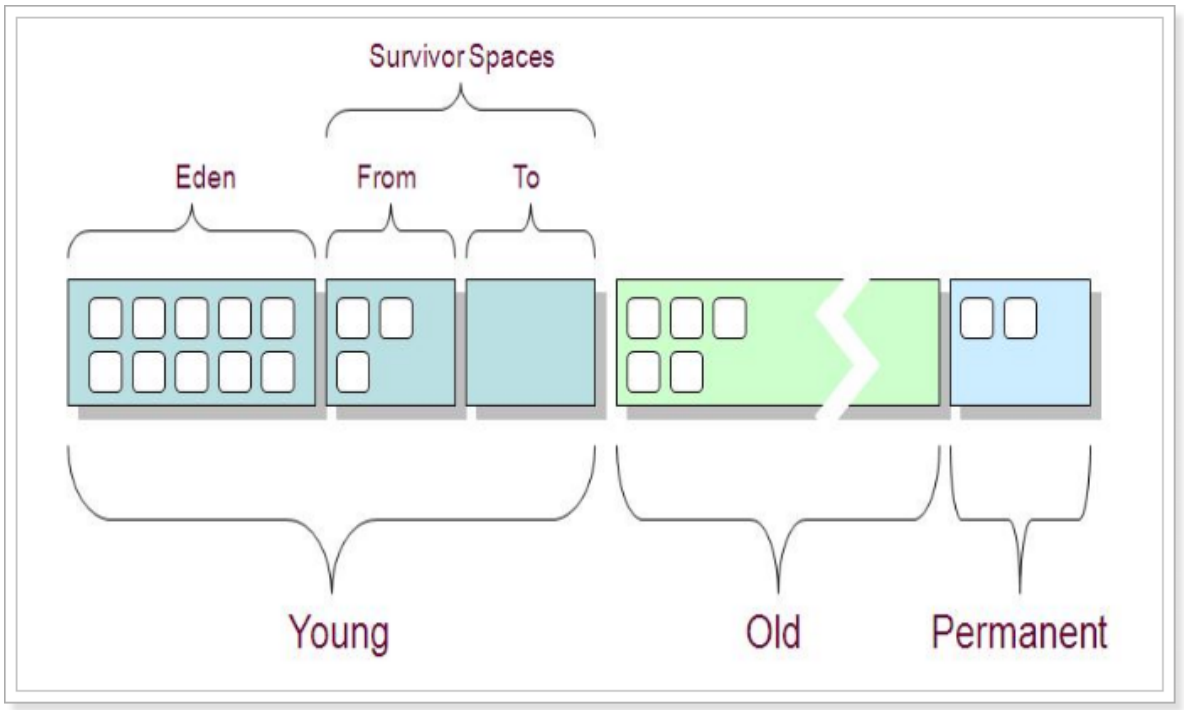
- 复制算法(Copying)
- 标记清除(Mark-Sweep)
- 标记压缩(Mark-Compact)，又称标记整理
- 分代收集算法(Generational-Collection)

4.1. JVM复习

JVM结构图：



堆内存结构：



GC的特点：

- 次数上频繁收集Young区
- 次数上较少收集Old区
- 基本不动Perm区

4.2. 垃圾判定

4.2.1. 引用计数法(Reference-Counting)

引用计数算法是通过判断对象的**引用数量**来决定对象是否可以被回收。

给对象中添加一个引用计数器，每当有一个地方引用它时，计数器值就加1；当引用失效时，计数器值就减1；任何时刻计数器为0的对象就是不可能再被使用的。

优点：

- 简单，高效，现在的objective-c、python等用的就是这种算法。

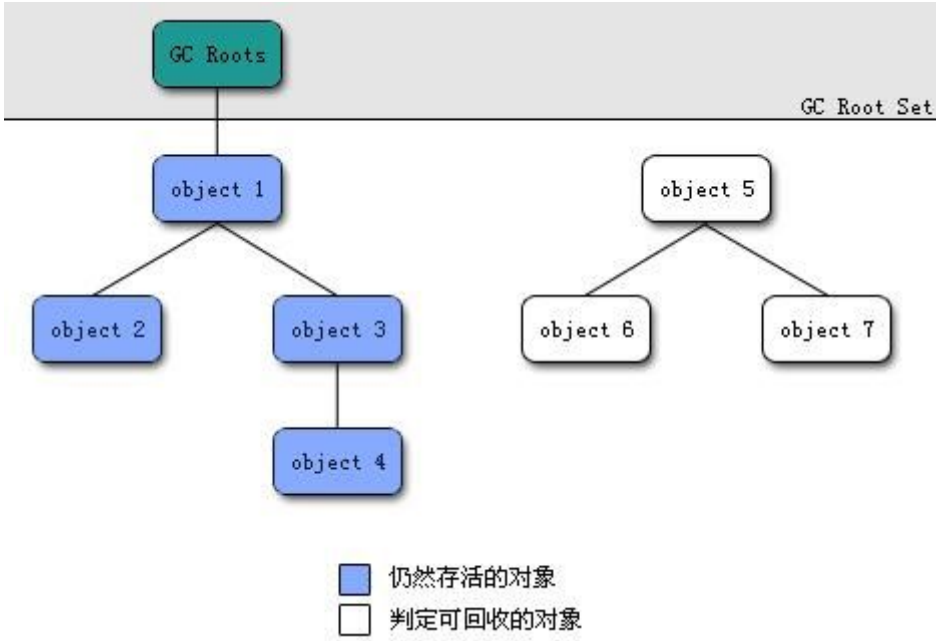
缺点：

- 引用和去引用伴随着加减算法，影响性能
- 很难处理循环引用，相互引用的两个对象则无法释放。

因此目前主流的Java虚拟机都摒弃掉了这种算法。

4.2.2. 可达性分析算法

这个算法的基本思想就是通过一系列的称为 **“GC Roots”** 的对象作为起点，从这些节点开始向下搜索，节点所走过的路径称为引用链，当一个对象到 GC Roots 没有任何引用链相连的话，则证明此对象是不可用的。



在Java语言中，可以作为GC Roots的对象包括下面几种：

- 虚拟机栈（栈帧中的本地变量表）中的引用对象。
- 方法区中的类静态属性引用的对象。
- 方法区中的常量引用的对象。
- 本地方法栈中JNI（Native方法）的引用对象

真正标记以为对象为可回收状态至少要标记两次。

4.2.3. 四种引用

平时只会用到强引用和软引用。

强引用：

类似于 Object obj = new Object(); 只要强引用还存在，垃圾收集器永远不会回收掉被引用的对象。

软引用：

SoftReference 类实现软引用。在系统要发生内存溢出异常之前，才会将这些对象列进回收范围之中进行二次回收。如果这次回收还没有足够的内存，才会抛出内存溢出异常。软引用可用来实现内存敏感的高速缓存。

弱引用：

WeakReference 类实现弱引用。对象只能生存到下一次垃圾收集之前。在垃圾收集器工作时，无论内存是否足够都会回收掉只被弱引用关联的对象。

虚引用：

PhantomReference 类实现虚引用。无法通过虚引用获取一个对象的实例，为一个对象设置虚引用关联的唯一目的就是能在这个对象被收集器回收时收到一个系统通知。

4.3. 垃圾回收算法

在介绍JVM垃圾回收算法前，先介绍一个概念：**Stop-the-World**

Stop-the-world意味着 JVM由于要执行GC而停止了应用程序的执行，并且这种情形会在任何一种GC算法中发生。当Stop-the-world发生时，除了GC所需的线程以外，所有线程都处于等待状态直到GC任务完成。事实上，**GC优化很多时候就是指减少Stop-the-world发生的时间，从而使系统具有高吞吐 、低停顿的特点。**

4.3.1. 复制算法(Copying)

该算法将内存平均分成两部分，然后每次只使用其中的一部分，当这部分内存满的时候，将内存中所有存活的对象复制到另一个内存中，然后将之前的内存清空，只使用这部分内存，循环下去。

优点:

- 实现简单
- 不产生内存碎片

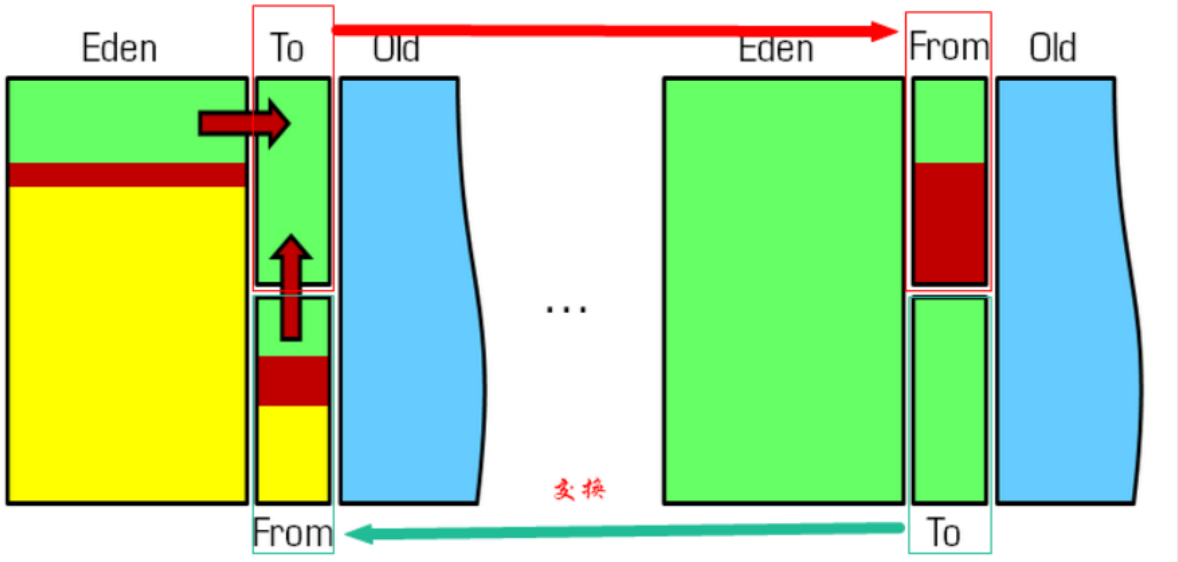
缺点:

- 将内存缩小为原来的一半，浪费了一半的内存空间，代价太高；如果不想浪费一半的空间，就需要有额外的空间进行分配担保，以应对被使用的内存中所有对象都100%存活的极端情况，所以在老年代一般不能直接选用这种算法。
- 如果对象的存活率很高，我们可以极端一点，假设是100%存活，那么我们需要将所有对象都复制一遍，并将所有引用地址重置一遍。复制这一工作所花费的时间，在对象存活率达到一定程度时，将会变的不可忽视。 所以从以上描述不难看出，复制算法要想使用，最起码对象的存活率要非常低才行，而且最重要的是，我们必须要克服50%内存的浪费。

年轻代中使用的是Minor GC，这种GC算法采用的是复制算法(Copying)。

HotSpot JVM把年轻代分为了三部分：1个Eden区和2个Survivor区（分别叫from和to）。默认比例为8:1:1,一般情况下，新创建的对象都会被分配到Eden区。因为年轻代中的对象基本都是朝生夕死的(90%以上)，所以在年轻代的垃圾回收算法使用的是复制算法。

在GC开始的时候，对象只会存在于Eden区和名为“From”的Survivor区，Survivor区“To”是空的。紧接着进行GC，Eden区中所有存活的对象都会被复制到“To”，而在“From”区中，仍存活的对象会根据他们的年龄值来决定去向。对象在Survivor区中每熬过一次Minor GC，年龄就会增加1岁。年龄达到一定值(年龄阈值，可以通过-XX:MaxTenuringThreshold来设置)的对象会被移动到年老代中，没有达到阈值的对象会被复制到“To”区域。经过这次GC后，Eden区和From区已经被清空。这个时候，“From”和“To”会交换他们的角色，也就是新的“To”就是上次GC前的“From”，新的“From”就是上次GC前的“To”。不管怎样，都会保证名为To的Survivor区域是空的。Minor GC会一直重复这样的过程，直到“To”区被填满，“To”区被填满之后，会将所有对象移动到年老代中。



因为Eden区对象一般存活率较低，一般的，使用两块10%的内存作为空闲和活动区间，而另外80%的内存，则是用来给新建对象分配内存的。一旦发生GC，将10%的from活动区间与另外80%中存活的eden对象转移到10%的to空闲区间，接下来，将之前90%的内存全部释放，以此类推。

4.3.2. 标记清除(Mark-Sweep)

“标记-清除”(Mark Sweep)算法是几种GC算法中最基础的算法，是因为后续的收集算法都是基于这种思路并对其不足进行改进而得到的。正如名字一样，算法分为2个阶段：

1. 标记出需要回收的对象，使用的标记算法均为**可达性分析算法**。
2. 回收被标记的对象。

缺点：

- 效率问题（两次遍历）
- 空间问题（标记清除后会产生大量不连续的碎片。JVM就不得不维持一个内存的空闲列表，这又是一种开销。而且在分配数组对象的时候，寻找连续的内存空间会不太好找。）

4.3.3. 标记压缩(Mark-Compact)

标记-整理法是标记-清除法的一个改进版。同样，在标记阶段，该算法也将所有对象标记为存活和死亡两种状态；不同的是，在第二个阶段，该算法并没有直接对死亡的对象进行清理，而是通过**所有存活对像都向一端移动，然后直接清除边界以外的内存。**

优点：

标记/整理算法不仅可以弥补标记/清除算法当中，内存区域分散的缺点，也消除了复制算法当中，内存减半的高额代价。

缺点：

如果存活的对象过多，整理阶段将会执行较多复制操作，导致算法效率降低。

老年代一般是由标记清除或者是标记清除与标记整理的混合实现。

4.3.4. 分代收集算法(Generational-Collection)

内存效率：复制算法>标记清除算法>标记整理算法（此处的效率只是简单的对比时间复杂度，实际情况不一定如此）。 内存整齐度：复制算法=标记整理算法>标记清除算法。 内存利用率：标记整理算法=标记清除算法>复制算法。

可以看出，效率上来说，复制算法是当之无愧的老大，但是却浪费了太多内存，而为了尽量兼顾上面所提到的三个指标，标记/整理算法相对来说更平滑一些，但效率上依然不尽如人意，它比复制算法多了一个标记的阶段，又比标记/清除多了一个整理内存的过程

难道就没有一种最优算法吗?

回答：无，没有最好的算法，只有最合适的算法。==========>**分代收集算法。**

分代回收算法实际上是把复制算法和标记整理法的结合，并不是真正一个新的算法，一般分为：老年代（Old Generation）和新生代（Young Generation），老年代就是很少垃圾需要进行回收的，新生代就是有很多的内存空间需要回收，所以不同代就采用不同的回收算法，以此来达到高效的回收算法。

年轻代(Young Gen)

年轻代特点是区域相对老年代较小，对像存活率低。

这种情况复制算法的回收整理，速度是最快的。复制算法的效率只和当前存活对像大小有关，因而很适用于年轻代的回收。而复制算法内存利用率不高的问题，通过hotspot中的两个survivor的设计得到缓解。

老年代(Tenure Gen)

老年代的特点是区域较大，对像存活率高。

这种情况，存在大量存活率高的对像，复制算法明显变得不合适。一般是由标记清除或者是标记清除与标记整理的混合实现。

4.4. 垃圾收集器（了解）

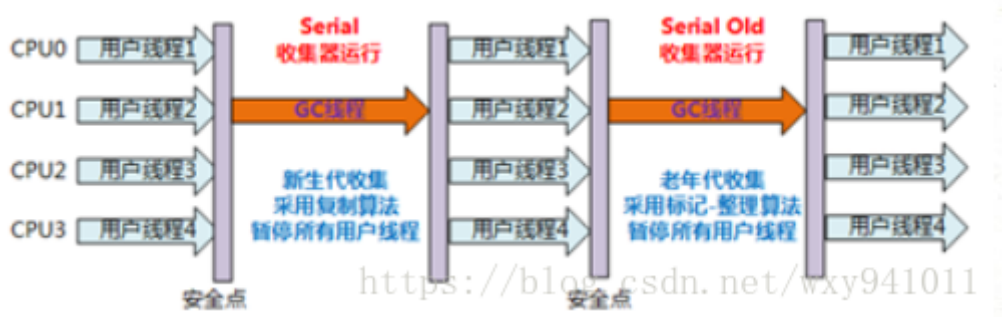
> 如果说收集算法是内存回收的方法论，垃圾收集器就是内存回收的具体实现

4.4.1. Serial/Serial Old收集器

串行收集器是最古老，最稳定以及效率高的收集器，可能会产生较长的停顿，只使用一个线程去回收。新生代、老年代使用串行回收；新生代复制算法、老年代标记-压缩；垃圾收集的过程中会Stop The World（服务暂停）

它还有对应老年代的版本：**Serial Old**

参数控制： `-XX:+UseSerialGC` 串行收集器



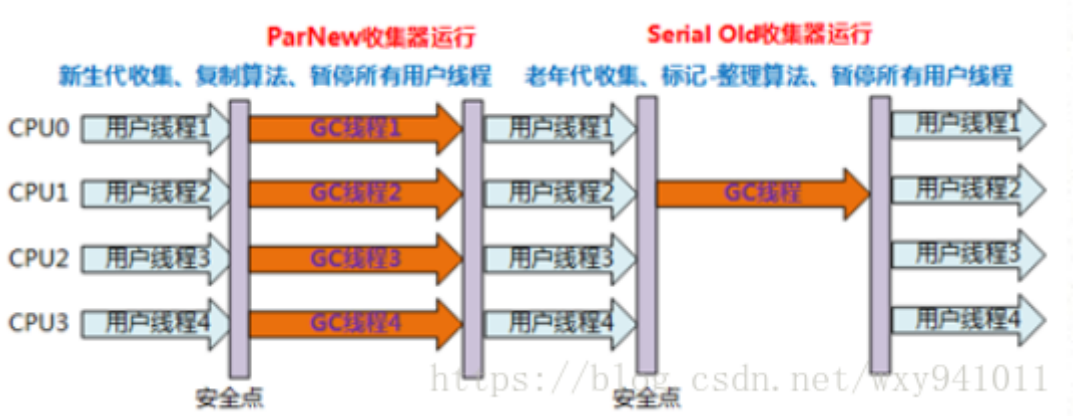
4.4.2. ParNew收集器

ParNew收集器是JAVA虚拟机中垃圾收集器的一种。它是Serial收集器的多线程版本，除了使用多条线程进行垃圾收集之外，其余行为包括Serial收集器可用的所有控制参数（例如：-XX:SurvivorRatio、-XX:PretenureSizeThreshold、-XX:HandlePromotionFailure等）、收集算法、Stop The World、对象分配规则、回收策略等都与Serial收集器一致。

ParNew是许多运行在Server模式下的虚拟机中首选的新生代收集器，除了Serial收集器外，只有它能与CMS收集器配合工作。

参数控制：

`-XX:+UseParNewGC` ParNew收集器 `-XX:ParallelGCThreads` 限制线程数量



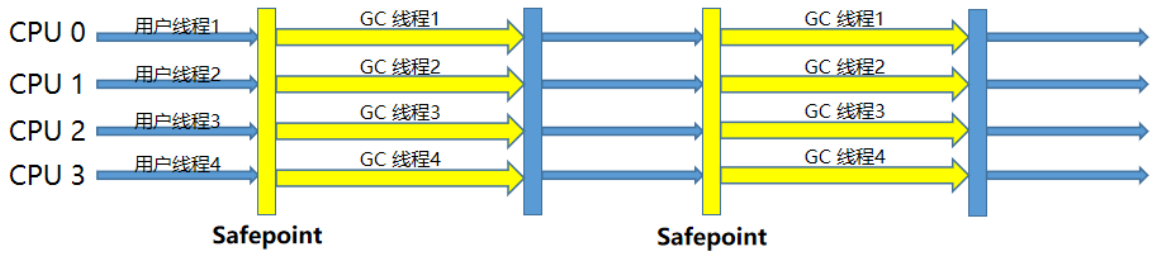
4.4.3. Parallel / Parallel Old收集器

Parallel Scavenge收集器类似ParNew收集器，Parallel收集器更关注系统的吞吐量。可以通过参数来打开自适应调节策略，虚拟机会根据当前系统的运行情况收集性能监控信息，动态调整这些参数以提供最合适的停顿时间或最大的吞吐量；也可以通过参数控制GC的时间不大于多少毫秒或者比例；新生代复制算法、老年代标记-压缩

参数控制： `-XX:+UseParallelGC` 使用Parallel收集器+ 老年代串行

Parallel Old是Parallel Scavenge收集器的老年代版本，使用多线程和“标记 - 整理”算法。这个收集器是在JDK 1.6中才开始提供

参数控制： `-XX:+UseParallelOldGC` 使用Parallel收集器+ 老年代并行



Parallel Scavenge+Parallel Old 收集器运行示意图

4.4.4. CMS收集器

CMS（Concurrent Mark Sweep）收集器是一种以获取最短回收停顿时间为目标的收集器。目前很大一部分的Java应用都集中在互联网站或B/S系统的服务端上，这类应用尤其重视服务的响应速度，希望系统停顿时间最短，以给用户带来较好的体验。

从名字（包含“Mark Sweep”）上就可以看出CMS收集器是基于“标记-清除”算法实现的，它的运作过程相对于前面几种收集器来说要更复杂一些，整个过程分为4个步骤，包括：

- 初始标记（CMS initial mark）
- 并发标记（CMS concurrent mark）
- 重新标记（CMS remark）
- 并发清除（CMS concurrent sweep）

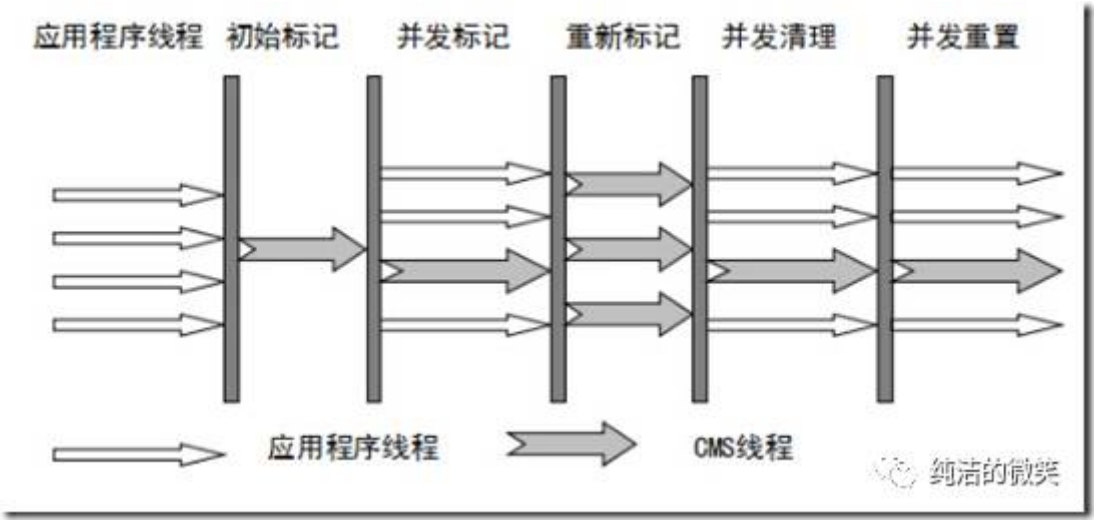
其中初始标记、重新标记这两个步骤仍然需要“Stop The World”。初始标记仅仅只是标记一下GC Roots能直接关联到的对象，速度很快，并发标记阶段就是进行GC Roots Tracing的过程，而重新标记阶段则是为了修正并发标记期间，因用户程序继续运作而导致标记产生变动的那一部分对象的标记记录，这个阶段的停顿时间一般会比初始标记阶段稍长一些，但远比并发标记的时间短。

由于整个过程中耗时最长的并发标记和并发清除过程中，收集器线程都可以与用户线程一起工作，所以总体上来说，CMS收集器的内存回收过程是与用户线程一起并发地执行。老年代收集器（新生代使用ParNew）

优点: 并发收集、低停顿 **缺点**: 产生大量空间碎片、并发阶段会降低吞吐量

参数控制：

`-XX:+UseConcMarkSweepGC` 使用CMS收集器 `-XX:+ UseCMSCompactAtFullCollection` Full GC后，进行一次碎片整理；整理过程是独占的，会引起停顿时间变长 `-XX:+CMSFullGCsBeforeCompaction` 设置进行几次Full GC后，进行一次碎片整理 `-XX:ParallelCMSThreads` 设定CMS的线程数量（一般情况约等于可用CPU数量）

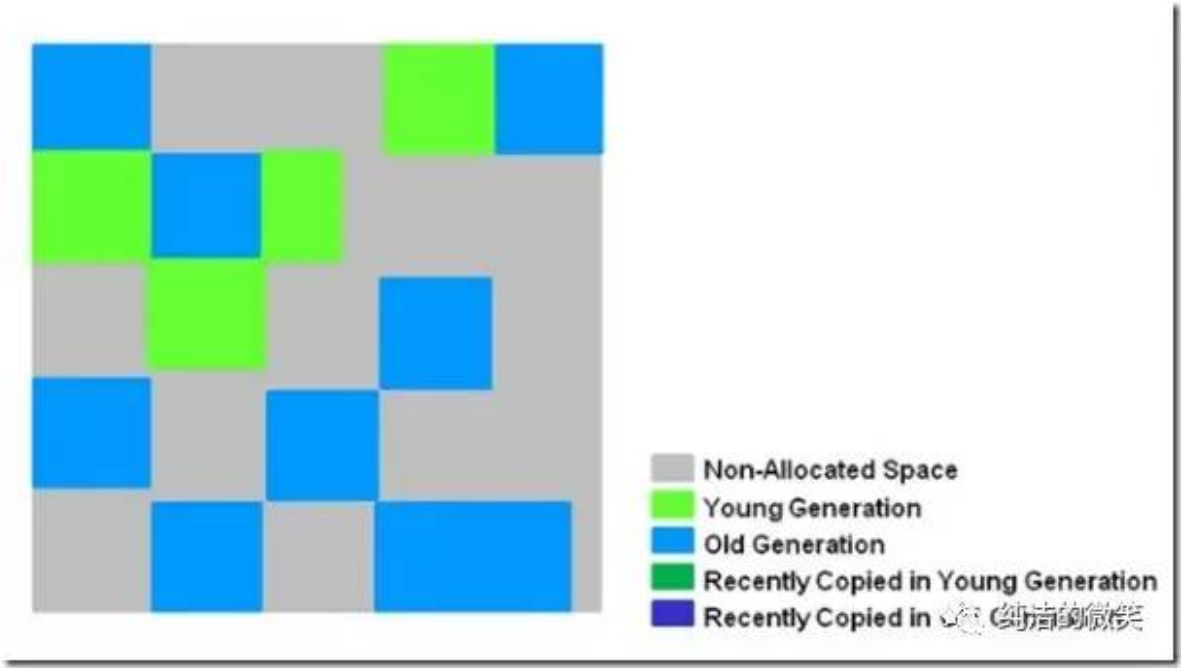


4.4.5. G1收集器

G1是目前技术发展的最前沿成果之一，HotSpot开发团队赋予它的使命是未来可以替换掉JDK1.5中发布的CMS收集器。与CMS收集器相比G1收集器有以下特点：

1. **空间整合**，G1收集器采用标记整理算法，不会产生内存空间碎片。分配大对象时不会因为无法找到连续空间而提前触发下一次GC。
2. **可预测停顿**，这是G1的另一大优势，降低停顿时间是G1和CMS的共同关注点，但G1除了追求低停顿外，还能建立可预测的停顿时间模型，能让使用者明确指定在一个长度为N毫秒的时间片段内，消耗在垃圾收集上的时间不得超过N毫秒，这几乎已经是实时Java（RTSJ）的垃圾收集器的特征了。

上面提到的垃圾收集器，收集的范围都是整个新生代或者老年代，而G1不再是这样。使用G1收集器时，Java堆的内存布局与其他收集器有很大差别，它将整个Java堆划分为多个大小相等的独立区域（Region），虽然还保留有新生代和老年代的概念，但新生代和老年代不再是物理隔阂了，它们都是一部分（可以不连续）Region的集合。



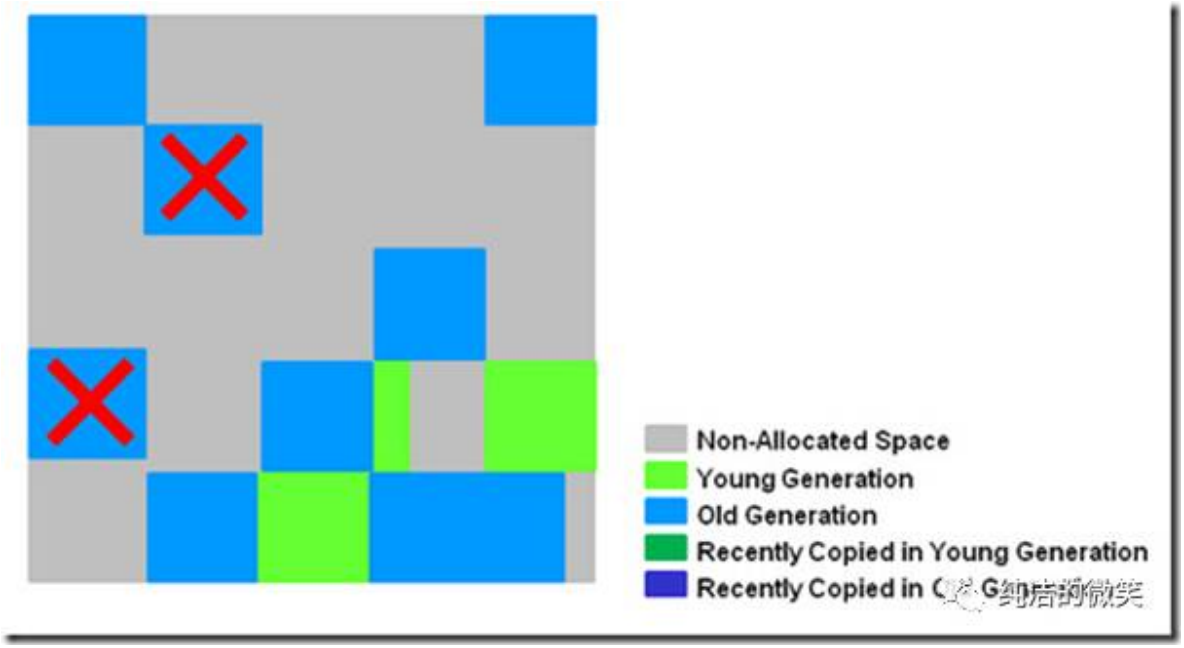
G1的新生代收集跟ParNew类似，当新生代占用达到一定比例的时候，开始触发收集。和CMS类似，G1收集器收集老年代对象会有短暂停顿。

收集步骤：

1、标记阶段，首先初始标记(Initial-Mark),这个阶段是停顿的(Stop the World Event)，并且会触发一次普通Mintor GC。对应GC log:GC pause (young) (inital-mark)

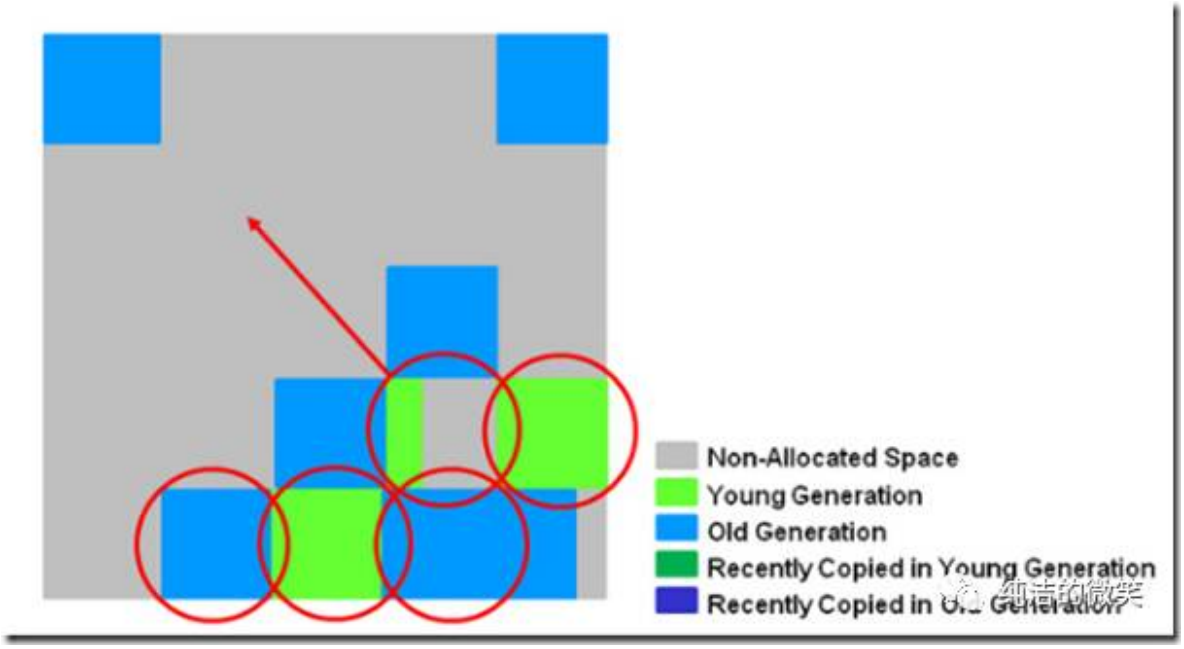
2、Root Region Scanning，程序运行过程中会回收survivor区(存活到老年代)，这一过程必须在young GC之前完成。

3、Concurrent Marking，在整个堆中进行并发标记(和应用程序并发执行)，此过程可能被young GC中断。在并发标记阶段，若发现区域对象中的所有对象都是垃圾，那个这个区域会被立即回收(图中打X)。同时，并发标记过程中，会计算每个区域的对象活性(区域中存活对象的比例)。

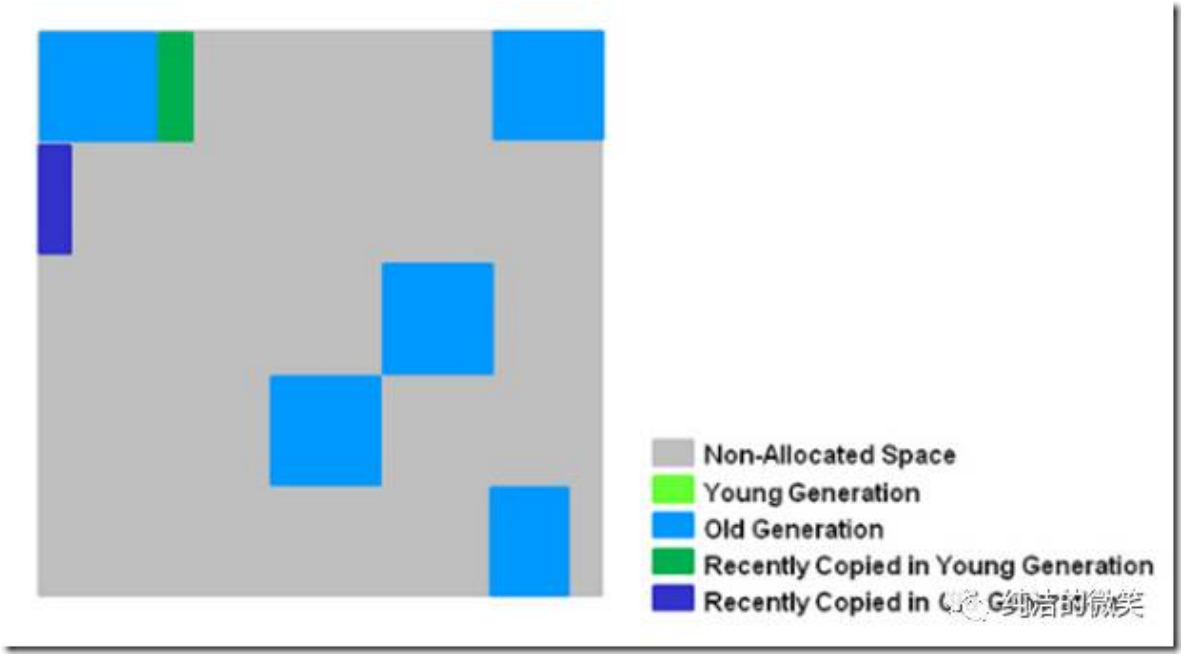


4、Remark, 再标记，会有短暂停顿(STW)。再标记阶段是用来收集 并发标记阶段 产生新的垃圾(并发阶段和应用程序一同运行)；G1中采用了比CMS更快的初始快照算法:snapshot-at-the-beginning (SATB)。

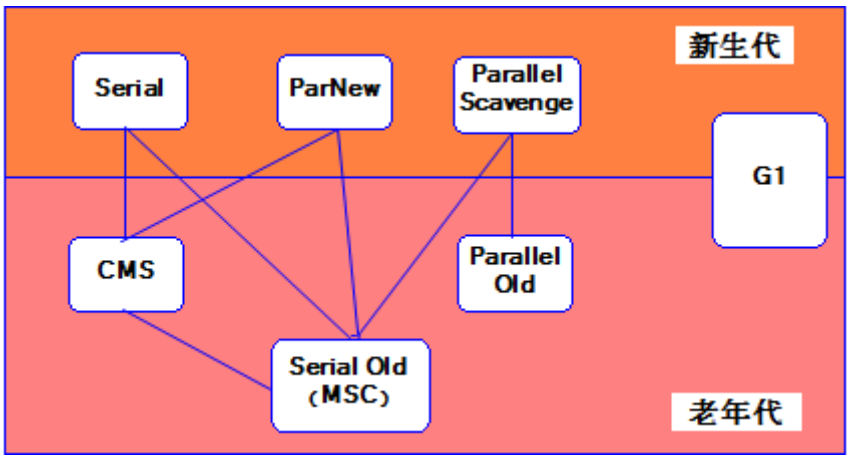
5、Copy/Clean up，多线程清除失活对象，会有STW。G1将回收区域的存活对象拷贝到新区域，清除Remember Sets，并发清空回收区域并把它返回到空闲区域链表中。



6、复制/清除过程后。回收区域的活性对象已经被集中回收到深蓝色和深绿色区域。



4.4.6. 垃圾回收器比较



如果两个收集器之间存在连线，则说明它们可以搭配使用。虚拟机所处的区域则表示它是属于新生代还是老年代收集器。

整堆收集器： G1

回收器名称	算法分类	作用区域	是否多线程	类型	备注
Serial	复制算法	新生代	单线程	串行	简单高效、不建议使用，Client默认
ParNew	复制算法	新生代	多线程	并行	唯一和CMS搭配的新生代垃圾回收器
Parallel Scavenge	复制算法	新生代	多线程	并行	更关注吞吐量
Serial Old	标记-整理	老年代	单线程	串行	能和所有young gc搭配使用
Parallel Old	标记-整理	老年代	多线程	并行	搭配Parallel Scavenge
CMS	标记-清除	老年代	多线程	并发	追求最短的暂停时间

知乎 @时雨旅客

垃圾回收器选择策略：

客户端程序： Serial + Serial Old；

吞吐率优先的服务端程序（比如：计算密集型）： Parallel Scavenge + Parallel Old；

响应时间优先的服务端程序： ParNew + CMS。

G1收集器是基于标记整理算法实现的，不会产生空间碎片，可以精确地控制停顿，将堆划分为多个大小固定的独立区域，并跟踪这些区域的垃圾堆积程度，在后台维护一个优先列表，每次根据允许的收集时间，优先回收垃圾最多的区域（Garbage First）。

我的

jvm的结构

类加载器

根加载器